



UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA

Corso di Laurea Magistrale in Informatica

TESI DI LAUREA

FedComp: A Framework for Model Compression Strategies in Federated Learning for IoT

RELATORI

Prof. Massimo Ficco

Dott. Antonio Guerriero

CANDIDATO

Andrea Bucci

Matricola: 0522501969

Anno Accademico 2025-2026

Se non hai paura, non corri nessun rischio. E se non corri nessun rischio allora che senso ha?

(Ted Mosby)

IoTResearch



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
DIPARTIMENTO DI ECCELLENZA

Abstract

Federated Learning (FL) has emerged as a prominent paradigm for distributed Machine Learning model training on edge devices, preserving local data privacy by design. However, the heterogeneous nature of participating clients, alongside strict memory, computational, and communication bandwidth constraints, poses significant operational challenges, particularly regarding the footprint of the models and network overhead during parameter synchronization.

To enable large-scale deployment across resource-constrained edge hardware, integrating Model Compression techniques within the federated lifecycle is essential. Nevertheless, compression methodologies, such as quantization, structured pruning, and knowledge distillation, do not exhibit uniform efficiency or effectiveness, as their performance strongly depends on underlying neural network architectures, data distributions, and target hardware platforms. Selecting an optimal strategy therefore requires rigorous, multi-dimensional benchmarking before physical deployment.

To address this challenge, this thesis introduces *FedComp*, a modular and reproducible evaluation framework designed to systematically analyze and compare the trade-offs of various model compression strategies embedded within a federated pipeline. Specifically, the framework enables the empirical evaluation of Quantization-Aware Training (QAT), Ordered Dropout, and Federated Knowledge Distillation, quantifying their impact on model convergence, predictive accuracy, and analytical resource metrics (bandwidth, FLOPs, and energy) across both IID and severe non-IID regimes.

As a practical follow-up to this empirical analysis, the most viable model compression technique is further exported and physically evaluated on real-world microcontroller hardware, validating theoretical gains against actual on-device execution constraints. By benchmarking these multi-dimensional trade-offs, this work establishes actionable guidelines for architecting sustainable, resource-aware Federated Learning systems prior to mass edge deployment.

Contents

I	Introduction	1
I.1	Problem Statement and Motivation	2
I.2	Thesis Objective and Framework Overview	2
I.3	Summary of Contributions	3
I.4	Thesis Structure	4
II	Background	6
II.1	Convolutional Neural Networks	6
II.1.1	Evolutionary Roots: From LeNet-5 to Modern CNNs	6
II.1.2	The Convolutional Layer	7
II.1.3	The Flattening and Classification Block	8
II.2	Federated Learning	10
II.2.1	Mathematical Formulation and the FedAvg Algorithm	11
II.2.2	Architectural Taxonomy: Cross-Silo vs. Cross-Device	12
II.2.3	Challenges in Federated IoT Environments	13
II.3	Model Compression Techniques in Resource-Constrained Environments	15
II.3.1	Knowledge Distillation	15
II.3.2	Network Pruning and Ordered Dropout	16
II.3.3	Quantization	17
III	Related Work	19
III.1	Model Compression Techniques	19
III.1.1	Evolution of Knowledge Distillation in Distributed Pipelines	19
III.1.2	Recent Advances and Trade-offs in Federated Quantization	21
III.1.3	Pruning Strategies for Heterogeneous Architecture Allocation	23
IV	Contribution	26

IV.1	Framework Overview	26
IV.2	System Architecture and General Workflow	26
IV.2.1	Shared Execution Backbone and Standardized Profiling	28
IV.2.2	General Federated Workflow	28
IV.3	An in-depth look at the components	29
IV.3.1	Baseline Standard FL	29
IV.3.2	Federated Knowledge Distillation	33
IV.3.3	Ordered Dropout	36
IV.3.4	Quantization-Aware Training (QAT) Extension	40
IV.4	Scientific and Technical Contributions	43
V	Experimentation	45
V.1	Experimental Datasets and Partitioning Regimes	45
V.1.1	Benchmark Datasets	45
V.1.2	Data Distribution Regimes: IID vs. Non-IID	46
V.2	Experimental Configuration and Hyperparameters	47
V.2.1	Network Architectures	47
V.2.2	Dataset-Specific Hyperparameter Profiles	48
V.2.3	Specific Framework Parameters	49
V.3	Resource Profiling and Telemetry Methodology	49
VI	Results	51
VI.1	Analysis on MNIST Dataset	51
VI.1.1	IID Regime Performance	51
VI.1.2	Non-IID Regime Performance ($\alpha = 0.1$)	55
VI.2	Analysis on Fashion-MNIST Dataset	59
VI.2.1	IID Regime Performance	59
VI.2.2	Non-IID Regime Performance ($\alpha = 0.1$)	62
VI.3	Analysis on FEMNIST Dataset	64
VI.3.1	IID Regime Performance	64
VI.3.2	Non-IID Regime Performance (Native Partitioning) . .	68
VI.4	Concluding Discussion and Trade-Off Analysis	72
VI.4.1	Strengths and Weaknesses of Evaluated Strategies . . .	72
VI.4.2	Identifying the Optimal Trade-Off Strategy	73
VII	Implementation on MCU	75
VII.1	System Architecture and Hardware Setup	75
VII.1.1	Physical Edge Nodes (ESP32-S3)	76

VII.1.2 Communication Protocol and Message Broker	77
VII.1.3 Centralized Distillation Coordinator	77
VII.2 Edge Firmware and On-Device Training	77
VII.2.1 Neural Architecture and Static Memory Allocation . . .	78
VII.2.2 Initialization and Symmetry Breaking	78
VII.2.3 Forward Pass and Numerical Stability	78
VII.2.4 Backpropagation and Stochastic Gradient Descent (SGD)	79
VII.2.5 Local Evaluation Pipeline	80
VII.3 Communication Protocol and Tensor Deserialization	80
VII.3.1 MQTT Transport Architecture and Topic Hierarchy . .	80
VII.3.2 Binary Parameter Serialization	81
VII.3.3 Reconciliation of Memory Layouts: Row-Major vs. Column-	
Major	82
VII.4 Server-Side Distillation and Global Evaluation	83
VII.4.1 Teacher Ensemble Reconstruction	83
VII.4.2 Proxy Dataset Ingestion	83
VII.4.3 Multi-Teacher Distillation Workflow	84
VII.4.4 Global Model Evaluation and Downlink Distribution .	84
VII.5 Hardware-Aware Profiling and Empirical Validation	85
VII.5.1 On-Device Computational Complexity and Execution	
Profiling	85
VII.5.2 Analytical Energy Consumption Model	86
VII.5.3 Empirical Accuracy and Convergence Analysis	87
VIII Conclusions	90

I Introduction

The rapid advancement of the Internet of Things (IoT) and the ubiquitous deployment of low-cost, resource-constrained embedded devices, commonly integrated into Microcontroller Units (MCUs), have fundamentally redefined the landscape of digital computing. Traditionally, Deep Learning (DL) models and Artificial Intelligence (AI) applications have been confined to high-performance Cloud infrastructures due to the massive computational overhead required by neural network training and inference. However, this fully centralized paradigm introduces significant structural bottlenecks, including prohibitive data transmission latency, an absolute reliance on continuous network connectivity, and critical data privacy vulnerabilities arising from the consolidation of sensitive user information.

To address these limitations, the scientific community has pioneered the migration of intelligence directly toward the network edge, giving rise to the emerging domain of TinyML. Concurrently, Federated Learning (FL) [1, 2] has emerged as a prominent distributed learning paradigm that enables edge nodes to collaboratively train a shared global model without exposing raw localized data. While enabling on-device training and local inference alleviates latency and privacy concerns, it introduces steep architectural and engineering challenges. Modern state-of-the-art Convolutional Neural Networks (CNNs) [3, 4] present memory footprints (both in volatile RAM and non-volatile Flash) and computational requirements (measured in FLOPs) that exceed the physical resource boundaries of commodity microcontrollers by several orders of magnitude.

To bridge this gap, *Model Compression* [5] has emerged as an essential field of study. Model compression aims to reduce the architectural complexity, parameter volume, and computational overhead of neural networks while preserving their empirical predictive capabilities. Within the context of Cross-Device Federated Learning [2], compressing local models does not merely optimize on-device inference; it serves as a foundational enabling mecha-

nism that drastically shrinks uplink and downlink communication payloads, allowing highly heterogeneous and bandwidth-constrained IoT nodes to participate in collaborative optimization cycles.

I.1 Problem Statement and Motivation

Despite the existence of several established compression methodologies, such as low-precision Quantization [6], structured and dynamic Pruning [7, 8], and Knowledge Distillation [9], their effectiveness and operational efficiency within federated edge topologies are far from uniform. The empirical performance of these techniques is intricately coupled with multiple conflicting variables, including the underlying neural architecture, the non-uniformity of local data partitions (IID vs. Non-IID regimes), task complexity (e.g., small-scale vs. massive multi-class label spaces), and the specific hardware constraints of the target microcontroller.

For instance, while low-precision quantization significantly reduces communication overhead, naive quantization-aware pipelines can suffer from gradient vanishing or stagnation under severe data scale, micro-batch regimes ($B = 1$), and non-differentiable approximations [10, 11, 12]. Similarly, structured pruning and sub-network slicing via Ordered Dropout [8] introduce elasticity across heterogeneous devices, yet evaluating narrower slices under heavy statistical skew can degrade global generalization. Consequently, selecting or designing an optimal compression strategy for real-world TinyML federated environments cannot rely on idealized assumptions; it mandates a rigorous, multi-dimensional empirical benchmarking framework capable of profiling trade-offs across predictive accuracy, communication bandwidth, local computational operations, and energy expenditure prior to physical deployment.

I.2 Thesis Objective and Framework Overview

To address these research challenges, this thesis presents *FedComp*, a modular, reproducible, and extensible experimental evaluation framework built on top of the *Flower* ecosystem [13]. *FedComp* is designed to systematically evaluate, benchmark, and compare the trade-offs of embedding core model compression paradigms directly into distributed federated learning pipelines under realistic edge conditions.

Specifically, the framework isolates and evaluates four distinct operational configurations:

- **Baseline Standard FL:** a full-precision (FP32) uncompressed federated architecture based on Federated Averaging (FedAvg) [1], serving as the foundational reference upper-bound for convergence and predictive accuracy;
- **Federated Knowledge Distillation (FKD):** an inverted distillation paradigm inspired by [9, 14], where clients train local compact architectures (`TinyNetIoT`) acting as Teachers, distilling soft ensemble logits to a global Student on the server over an unlabelled proxy dataset;
- **Ordered Dropout (OD):** a dynamic, elastic width-slicing mechanism inspired by FjORD [8] that structurally nests sub-networks within a single global architecture, allowing heterogeneous nodes to train capacity profiles ($p \in \{0.25, 0.5, 0.75, 1.0\}$) proportional to their local compute budgets;
- **Quantization-Aware Training (QAT):** an 8-bit low-precision pipeline simulating quantization noise during forward passes via Fake Quantization, while updating latent full-precision weights using the Straight-Through Estimator (STE) [6, 10].

Crucially, the framework embeds an analytical, literature-grounded profiling engine directly into the client runtime execution loop, enabling real-time telemetry tracking of communication bandwidth (B_{MB}), local inference FLOPs, and estimated energy expenditure (E_{total}) grounded in hardware literature [15, 16]. The evaluation spans standard benchmarks (MNIST [17], Fashion-MNIST [18]) as well as large-scale, naturally partitioned cross-device benchmarks (FEMNIST [19]). Furthermore, as a practical validation follow-up, the most viable compressed representations are ported and physically benchmarked on real-world microcontroller hardware, validating theoretical gains against actual embedded runtime execution.

I.3 Summary of Contributions

The main scientific and engineering contributions of this work are summarized as follows:

- **Design and Implementation of *FedComp*:** The architectural development of a modular, open-source benchmarking testbed providing strict statistical isolation, deterministic seeding, and shared pipelines across four federated compression strategies;
- **Multi-Dimensional Hardware-Aware Telemetry Engine:** The formulation and integration of an analytical IoT profiling methodology to capture granular communication footprints, computational overhead, and energy models during live client federated loops;
- **Comprehensive Multi-Regime Empirical Benchmarking:** An in-depth evaluation of compression resilience across both IID and severe non-IID data distributions ($\alpha = 0.1$ and native writer-based FEMNIST [19]), providing quantitative guidelines on trade-offs and identifying failure modes (such as QAT gradient stagnation under micro-batches);
- **Physical MCU Deployment and Validation:** The practical porting and execution of the resulting compressed models onto real-world microcontroller hardware to validate empirical simulation findings against embedded constraints.

I.4 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter II – Background:** provides the mathematical and theoretical foundations of Convolutional Neural Networks, the Federated Learning paradigm (FedAvg, taxonomy, and IoT challenges), and the principles of model compression (Knowledge Distillation, Ordered Dropout, and Quantization);
- **Chapter III – Related Work:** reviews contemporary literature on distributed distillation, federated quantization trade-offs, and dynamic pruning mechanisms for heterogeneous hardware allocation;
- **Chapter IV – Contribution:** details the software architecture, component design, dynamic communication workflows, and analytical resource telemetry engine powering the *FedComp* framework;

- **Chapter V – Experimentation:** outlines the experimental methodology, dataset specifications (MNIST, Fashion-MNIST, FEMNIST), partitioning regimes (IID vs. Non-IID), hyperparameter setups, and telemetry logging procedures;
- **Chapter VI – Results:** presents a comprehensive empirical analysis and comparative discussion of the results obtained across all benchmarks, highlighting the predictive and resource trade-offs of each compression strategy;
- **Chapter VII – Implementation on MCU:** details the follow-up experimental pipeline involving the porting, memory profiling, and physical execution of the compressed models on target microcontroller units;
- **Chapter VIII – Conclusions:** concludes the thesis by summarizing key takeaways, establishing design recommendations for edge federated systems, and highlighting avenues for future research.

II Background

This chapter provides the comprehensive theoretical foundation necessary to understand the methodologies leveraged in this thesis to enable efficient, distributed machine learning under strict hardware limitations. The remainder of this chapter is structured as follows:

- First, **Convolutional Neural Networks (CNNs)** are formally analyzed, dissecting the mathematical operations required for spatial feature extraction from visual signals;
- Next, the **Federated Learning (FL)** paradigm is introduced, with a specific analytical focus on optimization algorithms and stochastic convergence dynamics under micro-batch regimes ($B = 1$);
- Finally, the core principles of **Model Compression** are reviewed, deeply examining the mathematical mechanics of Quantization, Knowledge Distillation, and structured model pruning via Ordered Dropout, establishing them as essential frameworks to optimize the trade-off between predictive accuracy and hardware sustainability.

II.1 Convolutional Neural Networks

II.1.1 Evolutionary Roots: From LeNet-5 to Modern CNNs

The foundation of modern computer vision relies heavily on Convolutional Neural Networks, a specialized class of deep neural networks designed to process data possessing a known, grid-like topology, such as two-dimensional images. Prior to the widespread adoption of CNNs, traditional Multi-Layer Perceptrons (MLPs) or fully connected networks were the standard paradigm for pattern recognition tasks. However, scaling MLPs to high-resolution visual inputs introduces critical structural limitations. In a fully connected layer, every input pixel is mapped to every neuron in

the subsequent layer, resulting in a catastrophic explosion in the number of trainable parameters. This spatial unrolling discards the intrinsic two-dimensional spatial correlations present in images, rendering the network highly sensitive to translations, scaling, and rotations of the input features, while simultaneously increasing the risk of overfitting.

To overcome these scalability and structural bottlenecks, Yann LeCun et al. introduced LeNet-5 in their seminal 1998 paper [3]. Inspired by early neurobiological models of the human visual cortex (specifically the concepts of simple and complex cells discovered by Hubel and Wiesel), LeNet-5 established the foundational architectural blueprint for modern CNNs. The primary innovation of this framework was the shift from global connectivity to localized feature extraction, alternating convolutional layers with spatial subsampling (pooling) layers, before culminating in fully connected classification layers.

LeNet-5 demonstrated groundbreaking performance on the task of handwritten digit recognition using the MNIST dataset, proving that a deep learning model could successfully achieve local shift, scale, and distortion invariance. By enforcing three architectural principles (local receptive fields, shared weights, and spatial subsampling), this pioneer network proved that capturing hierarchical local features is computationally superior and hardware-friendlier than computing global relationships, laying the groundwork for contemporary embedded machine learning and TinyML applications.

II.1.2 The Convolutional Layer

The mathematical core of a CNN is the convolutional layer, which replaces the standard matrix multiplication found in traditional dense networks with a specialized linear operation known as discrete convolution [4]. In computer vision applications, the input is typically structured as a three-dimensional tensor $I \in \mathbb{R}^{H \times W \times C}$, where H , W , and C denote the height, width, and number of channels, respectively (e.g., $C = 1$ for grayscale datasets or $C = 3$ for RGB datasets).

Instead of mapping every input pixel to every single output neuron, this layer slides a small, learnable matrix called a *kernel* or *filter* (K) across the spatial dimensions of the input. At each localized position, an element-wise multiplication between the kernel weights and the overlapping patch of the input image is computed, followed by a summation and the addition of a

scalar bias term b . Mathematically, the value of a single pixel in the output feature map S at spatial coordinates (i, j) is defined as:

$$S(i, j) = \sum_c \sum_m \sum_n I(i \cdot s + m, j \cdot s + n, c) \cdot K(m, n, c) + b \quad (\text{II.1.1})$$

where s represents the *stride*, a hyperparameter regulating the step size of the filter as it shifts across the image grid.

According to Goodfellow et al. [4], the structural and computational efficiency of this operation over traditional fully connected layers stems from two foundational properties:

- **Sparse Interactions:** unlike dense layers where every input unit interacts with every output unit, CNN units exhibit a localized receptive field. A single neuron in a feature map is connected only to a small, local sub-region of the input pixels determined by the kernel dimensions. This significantly reduces runtime memory allocation and lowers the required number of floating-point operations (FLOPs);
- **Parameter Sharing:** in a traditional architecture, each element of the weight matrix is used exactly once to calculate a single output unit. In contrast, a convolutional layer reuses the same set of kernel weights systematically across the entire input grid to construct the feature map. This enforces the structural assumption that a visual feature discovered in one part of an image (such as an edge) is equally meaningful in other spatial locations.

By enforcing parameter sharing and localized receptive fields, the convolutional layer dramatically slims down the model's footprint. This properties are crucial for deployment on resource-constrained hardware, directly preventing memory saturation during training and inference on Microcontroller Units (MCUs).

II.1.3 The Flattening and Classification Block

The final phase of a Convolutional Neural Network architecture transitions the model from spatial feature extraction to categorical classification. After visual signals propagate through the alternating sequence of convolutional and pooling layers, the resulting output is encapsulated within a high-level, multi-dimensional tensor.

Because traditional dense, fully connected layers (*Fully Connected*) are mathematically defined to accept exclusively one-dimensional vector arrays, a specialized translation operation known as *Flattening* must be executed. The flattening layer acts as a structural bridge; it unrolls the multi-dimensional feature tensor, systematically aligning every individual pixel value into a contiguous, single-dimensional vector.

Once the spatial attributes are linearized, they are fed into the classification block, which consists of a series of dense layers [4]. Within these layers, every input node is connected to every subsequent neuron, enabling the network to compute complex global combinations of the extracted local features. The final linear layer maps these internal representations to a vector of raw unnormalized scores, designated as *logits*.

To interpret these logits as verifiable classification outcomes, the network applies the *Softmax* activation function. For a given logit vector z , the probability P of the input sample belonging to a specific class i out of a total K classes is mathematically formalized as:

$$P(y = i \mid z) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (\text{II.1.2})$$

The Softmax operation effectively normalizes the raw exponential outputs into a categorical probability distribution summing to 1. This mathematical framework allows the server and edge clients to evaluate predictions rigorously, providing the foundational metrics necessary to compute cross-entropy loss functions and steer backpropagation routines.

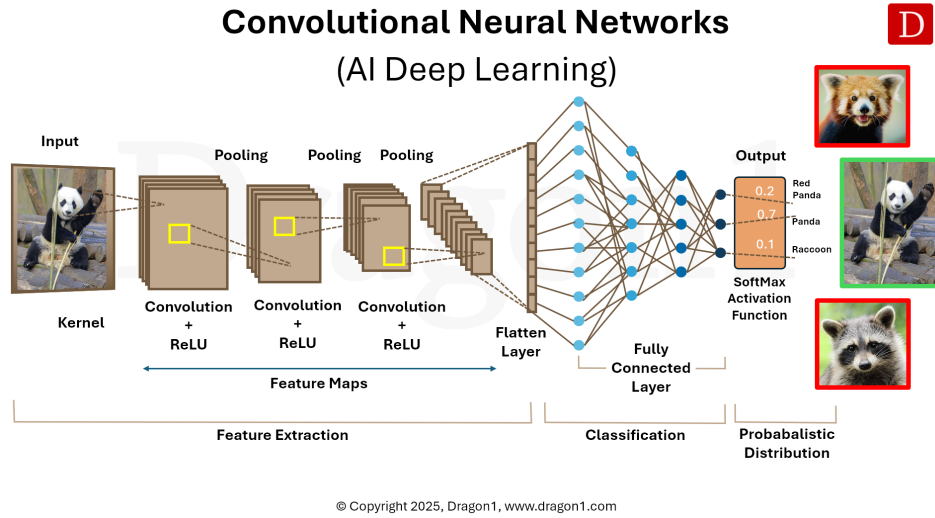


Figure II.1: Structural pipeline of a Convolutional Neural Network (CNN), illustrating the transition from spatial feature maps through the flattening layer to the fully connected classification block. *[Image courtesy of Dragon1]*

II.2 Federated Learning

Federated Learning (FL) represents a paradigm shift in the field of distributed Machine Learning. Originally introduced by McMahan et al.[1], FL shifts the training of predictive models directly to where data are generated, reversing the classical centralized machine learning approach. Instead of transmitting massive amounts of raw data to a centralized Cloud infrastructure, the federated framework distributes the global model parameters to participating nodes (*clients*). These clients execute local optimization and subsequently transmit only the parameter updates (either gradients or weights) back to a central coordinator or server. This approach natively addresses data privacy concerns, substantially reduces network bandwidth consumption, and mitigates the inherent risks associated with data centralization.

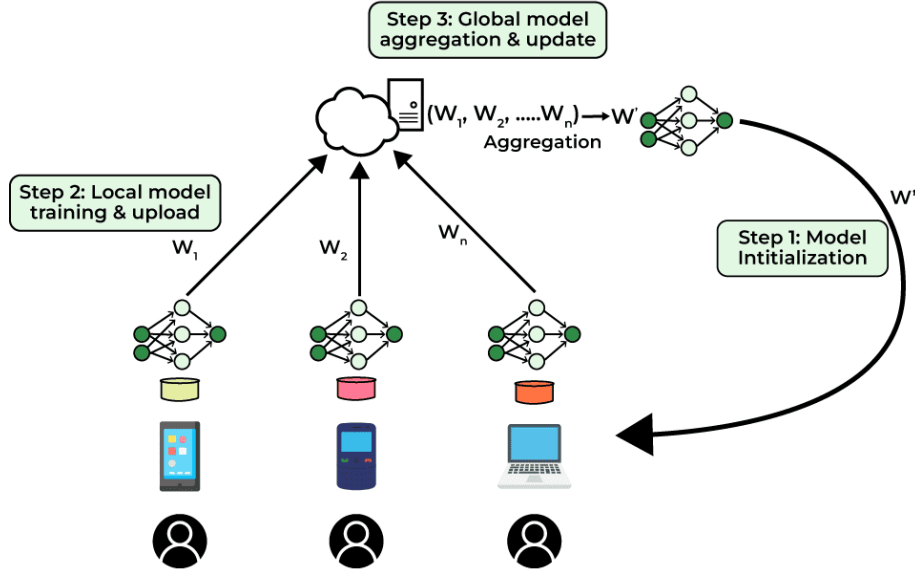


Figure II.2: Typical workflow of Federated Learning. [Image courtesy of GeeksForGeeks]

II.2.1 Mathematical Formulation and the FedAvg Algorithm

The federated learning problem can be formalized as a distributed optimization task aimed at minimizing a global objective loss function across a total of K clients. The standard mathematical formulation is expressed as:

$$\min_{w \in \mathbb{R}^d} F(w) \quad \text{where} \quad F(w) := \sum_{k=1}^K \frac{n_k}{n} F_k(w) \quad (\text{II.2.1})$$

In this formulation, $w \in \mathbb{R}^d$ denotes the global model parameter vector of dimension d , n_k represents the number of data samples available locally at client k , and $n = \sum_{k=1}^K n_k$ is the total cardinality of the distributed dataset. The local objective function for the k -th client, $F_k(w)$, is defined as the empirical risk over its local data distribution $\mathcal{D}_k$, computed via a loss function $\ell(x_i, y_i; w)$ on individual input-output pairs (x_i, y_i) :

$$F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{D}_k} \ell(x_i, y_i; w) \quad (\text{II.2.2})$$

The cornerstone algorithm for solving this optimization objective is *Federated Averaging* (FedAvg) [1]. The protocol operates iteratively through a series of communication rounds synchronized by the central server. At each round t :

1. The server selects a fraction C (where $0 < C \leq 1$) of available clients to

participate in the current round, denoted as the subset $\mathcal{S}_t$.

2. The server broadcasts the current global model state w_t to all clients included in $\mathcal{S}_t$.
3. Each selected client executes E epochs of local training using its local dataset and an optimizer, such as Stochastic Gradient Descent (SGD) with a learning rate η , updating its local weights:

$$w_{t+1}^k \leftarrow w_t - \eta \nabla F_k(w_t) \quad (\text{II.2.3})$$

4. The participating clients transmit their updated local parameters w_{t+1}^k back to the central server.
5. The server aggregates the received parameters by computing a weighted average proportional to each client's dataset size:

$$w_{t+1} \leftarrow \sum_{k \in \mathcal{S}_t} \frac{n_k}{\sum_{j \in \mathcal{S}_t} n_j} w_{t+1}^k \quad (\text{II.2.4})$$

II.2.2 Architectural Taxonomy: Cross-Silo vs. Cross-Device

Depending on the nature of the participating entities, data volume, and network characteristics, the literature [2] categorizes FL systems into two distinct macro-architectures: *Cross-Silo* and *Cross-Device*. This taxonomy is critical as it fundamentally defines the operational constraints of the entire system.

Cross-Silo Federated Learning

In a Cross-Silo scenario, clients are stable institutional or commercial organizations, such as financial consortiums, medical centers, or corporate branches. The total number of participants is typically small, ranging from a few units to several dozen. The computational resources at each node are substantial (e.g., dedicated servers or GPU clusters), and the communication channels are characterized by high reliability, high bandwidth, and low latency. In this setting, almost all clients participate in every single computation round, and bottlenecks are primarily associated with regulatory compliance, multi-party orchestration, or algorithmic design, rather than hardware limitations.

Cross-Device Federated Learning and the IoT Ecosystem

Conversely, the Cross-Device framework applies to massively distributed networks, which are highly representative of IoT deployment scenarios and mobile edge networks. Here, clients consist of thousands or millions of edge devices, such as smartphones, smart home appliances, microcontrollers, and embedded sensor nodes. This environment presents radically different challenges compared to Cross-Silo FL:

- **Scale and Ephemeral Availability:** only a tiny fraction of the client pool is active and eligible to participate in any given round t . Devices connect intermittently, often depending on whether they are charging, unmetered, or connected to Wi-Fi networks, which introduces severe client straggler and drop-out phenomena;
- **Rigid Hardware Constraints:** Edge IoT nodes operate under strict physical limitations, including highly constrained volatile RAM, limited non-volatile storage, low-power CPU/MCU architectures, and finite battery reserves;
- **Communication Bottlenecks:** uplink transmission (from client to server) is highly asymmetric, energy-costly, and prone to failures. Constantly transferring large Deep Learning model architectures over wireless links is unsustainable under standard edge operational conditions.

Table II.1 highlights the structural and operational differences between these two architectures.

II.2.3 Challenges in Federated IoT Environments

Integrating Cross-Device FL within IoT architectures exposes three foundational forms of heterogeneity that jeopardize global model convergence and implementation feasibility:

1. **Statistical Heterogeneity (Non-IID Data):** the local data distributions across different clients ($\mathcal{D}_k$) are typically Non-Identically and Independently Distributed (*Non-IID*). User behavior patterns or distinct environmental deployments of sensors introduce massive class imbalances, feature distribution shifts, and concept drift. This causes *client drift*, where local optimizers push local parameters toward diverging

Table II.1: Taxonomical comparison between Cross-Silo and Cross-Device Federated Learning architectures.

Feature	Cross-Silo FL	Cross-Device FL (IoT)
Client Distribution	Few institutional nodes (2–100)	Massive population (10^3 – 10^9)
Network Reliability	High reliability, symmetric bandwidth	Unstable channels, restricted bandwidth
Node Availability	Almost always online (100%)	Ephemeral, contingent on external factors
Compute Power	High (Data Centers, GPU clusters)	Highly constrained and heterogeneous (Edge, MCU)
Primary Bottleneck	Algorithmic / Regulatory compliance	Communication bandwidth and hardware resources

minima, severely degrading the accuracy of the aggregated global model under standard aggregation protocols;

2. **System Heterogeneity:** IoT devices in real-world deployments are inherently non-homogeneous. High-performance Edge devices (e.g., Single Board Computers like Raspberry Pi) coexist with ultra-low-power microcontrollers (e.g., ARM Cortex-M based MCU architectures). Modern Convolutional Neural Networks (CNNs) require tens of megabytes to store weights and millions of floating-point operations (FLOPs) for forward and backward passes, exceeding the physical limits of resource-constrained IoT nodes by multiple orders of magnitude;
3. **Network Heterogeneity and Channel Costs:** over-the-air parameter transmission quickly saturates the available communication bandwidth of standard IoT protocols.

The intersection of extreme system heterogeneity and strict communication bottlenecks defines the precise requirement for integrating **Model Compression** techniques directly into the federated training lifecycle. Reducing the memory footprint of local models and lightening the transmission payload is no longer a mere performance optimization; it is an enabling requirement to allow constrained IoT nodes to participate in collaborative learning.

II.3 Model Compression Techniques in Resource-Constrained Environments

Despite the remarkable capabilities of Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) across various computer vision and edge computing tasks, their implementation on IoT nodes is severely limited by their substantial demand for computational and memory resources. Modern state-of-the-art architectures frequently encompass tens of millions of parameters, requiring substantial volatile memory (RAM) for runtime execution and floating-point execution capabilities that far exceed the physical boundaries of commodity microcontrollers (MCUs).

To bridge this gap, *Model Compression* has emerged as a crucial field of study. Model compression aims to reduce the architectural complexity, storage requirements, and computational density of deep models while preserving their empirical generalization accuracy. Within the context of Cross-Device Federated Learning, compressing the local model does not merely optimize inference performance at the edge; it serves as an enabling mechanism that reduces the upstream and downstream communication payload, allowing highly heterogeneous nodes to participate in collaborative training cycles.

II.3.1 Knowledge Distillation

Knowledge Distillation (KD), formalised in the seminal work by Hinton et al. [9], is a compression paradigm where a compact, lightweight model (the *student*) is trained to replicate the behavioral characteristics and functional mappings of a massive, highly parameterized pre-trained network (the *teacher*).

Instead of training the student network exclusively on hard targets (i.e., one-hot encoded ground truth labels), KD leverages the "dark knowledge" embedded within the continuous probability distributions generated by the teacher's soft targets. This is achieved by introducing a hyperparameter called *Temperature* (T) into the standard softmax activation function. The softened softmax output for the i -th class logit z_i is defined as:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (\text{II.3.1})$$

When $T = 1$, the formulation reverts to the standard softmax operation. A higher temperature value smooths the distribution, exposing the relative probabilities that the teacher assigns to incorrect classes (e.g., how much a specific input image of a dog resembles a cat versus a car).

The optimization objective for the student network is governed by a combined loss function consisting of a traditional cross-entropy loss ($\mathcal{L}_{CE}$) with respect to the hard ground truth labels, and a Kullback-Leibler divergence loss ($\mathcal{L}_{KL}$) that minimizes the statistical distance between the soft logit distributions of the student and the teacher:

$$\mathcal{L}_{KD} = (1 - \alpha)\mathcal{L}_{CE}(y, \sigma(z_s)) + \alpha T^2 \mathcal{L}_{KL}(\sigma(z_t/T), \sigma(z_s/T)) \quad (\text{II.3.2})$$

where σ denotes the softmax function, z_s and z_t represent the logits of the student and teacher respectively, and α is a balancing coefficient. In a federated framework, KD is often employed to stabilize convergence under heavily non-IID data distributions or to allow clients to train localized, heterogeneous student architectures while aggregating generalized knowledge on a server-side teacher.

II.3.2 Network Pruning and Ordered Dropout

Network pruning operates on the premise that deep neural networks are intrinsically over-parameterized, containing a high degree of redundant weights that contribute negligibly to the final inference accuracy. Originally introduced conceptually by LeCun et al. [7] via "Optimal Brain Damage", modern pruning strategies systematically eliminate non-essential parameters based on predefined saliency criteria (such as absolute magnitude or gradient sensitivity). Pruning is broadly categorized into two approaches:

- **Unstructured Pruning:** individual weights below a specific threshold are zeroed out. While this significantly drops the theoretical parameter count, it creates sparse weight matrices that require specialized hardware accelerators or libraries to yield actual runtime speedups;
- **Structured Pruning:** entire architectural components, such as convolutional channels, filters, or complete layers, are excised. This directly translates to physical memory reduction and computational speedups

on standard commodity IoT hardware without requiring sparse computing frameworks.

A highly relevant, specialized implementation of structural pruning in federated environments is *Ordered Dropout* (OD), introduced as a core mechanism within the FjORD framework by Horvath et al. [8]. Unlike standard dropout, which randomly deactivates neurons to prevent overfitting, Ordered Dropout enforces a strict, deterministic prioritization over the network's parameters.

By systematically dropping parameters starting from the rightmost columns or lower-priority structural components, OD enables a single global model architecture to dynamically instantiate nested submodels (*ordered subnetworks*) of varying widths. High-performance edge nodes can train the full-width model, whereas extremely constrained microcontrollers train a compact, highly pruned subnetwork composed only of the highest-priority left-most parameters. Upon aggregation via a modified federated protocol, these overlapping nested updates are combined, natively resolving system heterogeneity without requiring distinct architecture instances.

II.3.3 Quantization

Quantization reduces the physical storage footprint and computational complexity of DNNs by mapping continuous, high-precision floating-point parameter representations (typically 32-bit IEEE floating-point, FP32) into discrete, lower-bitwidth numeric formats, such as 8-bit integers (INT8) or even binary/ternary representations. The definitive theoretical framework and practical review of quantization strategies for edge devices are detailed by Krishnamoorthi [6].

Quantization is primarily divided into two execution workflows:

1. **Post-Training Quantization (PTQ):** the model is trained entirely using standard FP32 precision. Once convergence is achieved, a calibration dataset is passed through the network to calculate the dynamic ranges of weights and activations, mapping them directly to fixed-point INT8 values. PTQ requires minimal computational overhead but can lead to severe accuracy degradation in highly sensitive architectures like compact CNNs;
2. **Quantization-Aware Training (QAT):** to mitigate the precision loss of PTQ, QAT models quantization constraints directly during the training

phase. Rather than optimizing the model natively in a low-bitwidth domain (which destabilizes gradient descent), QAT introduces a mechanism known as *Fake Quantization*. The training workflow is structured as follows:

- **Forward Pass (Simulated Rounding):** the full-precision (FP32) weights and activations are temporarily converted into discrete low-precision target values (e.g., INT8) and immediately cast back to FP32. This allows the network to execute forward computations while experiencing the exact quantization noise and resolution loss that will occur during edge deployment;
- **Backward Pass (Straight-Through Estimator):** since the rounding and clipping operations used in quantization are step functions, their mathematical derivatives are zero everywhere (or undefined at the steps), which would inherently halt backpropagation. To bypass this, QAT adopts the *Straight-Through Estimator* (STE) algorithm [10]. During the backward pass, the optimizer treats the rounding operation as an identity function, ignoring its derivative and passing the gradients directly to the latent full-precision (FP32) weights, which are updated normally;
- **Final Conversion:** upon completing the training cycle, the fake quantization modules are removed, and the optimized parameters are permanently converted and stored in the low-bitwidth format for production-ready deployment.

In Cross-Device Federated Learning, applying quantization to the model parameters or the calculated gradients dramatically accelerates the training process. It compresses the uplink and downlink payloads by approximately 75% (when converting from FP32 to INT8), mitigating communication bottlenecks while enabling energy-efficient INT8 matrix multiplication on low-power edge hardware.

III Related Work

The deployment of deep learning models in distributed environments has revolutionized the field of artificial intelligence, enabling localized training without compromising data privacy. However, deploying state-of-the-art neural networks within resource-constrained Internet of Things ecosystems poses significant challenges due to the strict limitations of edge hardware:

- Limited memory capacity, which prevents the storage of massive parameter matrices;
- Restricted computational capabilities, which prolong local training phases and deplete battery life;
- Constrained communication bandwidth, which leads to network bottlenecks during the transmission of heavy model updates.

To mitigate these hardware-induced limitations, the research community has extensively investigated model compression methodologies. These techniques aim to reduce the architectural footprint of neural networks while retaining their generalization capabilities. This chapter provides a comprehensive review of the state of the art in model compression, analyzing techniques such as Knowledge Distillation, Pruning and Quantization, before exploring their evolution into heterogeneous and dynamic federated learning environments.

III.1 Model Compression Techniques

III.1.1 Evolution of Knowledge Distillation in Distributed Pipelines

While the foundational mathematical framework of Knowledge Distillation (KD) focuses on transferring parameters from a static teacher to a smaller

student [9], modern literature has redefined its role within distributed and collaborative learning paradigms. In particular, recent developments have extended the centralized formulation by Hinton et al. [9] to address the specific challenges of communication overhead and architectural constraints in edge networks [8, 20].

To fully contextualize this paradigm shift, it is essential to delineate the operational divergence between traditional and contemporary distillation structures, which can be categorized into two primary training regimes:

- **Offline knowledge distillation:** which follows a strictly sequential, multi-stage execution where the complex teacher network must be completely pre-trained and its structural parameters frozen before the student training phase can begin [5, 9];
- **Online knowledge distillation:** commonly referred to as co-distillation, which eliminates the requirement of a pre-trained oracle by forcing both networks to evolve simultaneously in a collaborative fashion, acting bidirectionally as both teachers and students at every training step [5, 21].

A major focus of recent state-of-the-art research involves shifting away from offline distillation protocols, which mandate a multi-stage training pipeline where the teacher network must be fully optimized prior to student deployment. To eliminate this operational bottleneck, several architectures have introduced online knowledge distillation, also known as co-distillation [21]:

- **Peer-to-peer ensembles:** where multiple compact networks train simultaneously and exchange localized representations;
- **Mutual learning strategies:** which force distinct student topologies to stabilize prediction boundaries iteratively without relying on a static, high-parameter oracle [21];
- **Deep mutual distillation frameworks:** where structural knowledge is distilled bidirectionally across layers of varying depths throughout the entire optimization process [5].

Within the context of federated pipelines, the intersection of KD and model compression has led to the emergence of **Federated Distillation (FD)**

protocols. Traditional federated frameworks require clients to upload entire parameter matrices, creating significant communication bottlenecks on resource-constrained devices [20]. Conversely, contemporary studies leverage distillation to decouple local training from global aggregation:

- **Logit-based aggregation schemes:** where clients transmit exclusively their non-normalized output vectors computed over an auxiliary proxy dataset;
- **Ensemble distillation on the server side:** which enables the global model to aggregate insights from structurally heterogeneous client networks by averaging their generated teacher logits over a proxy dataset, thereby eliminating the need to perform an algebraic weight averaging [14];
- **Data-free federated distillation methods:** which employ generative adversarial setups on the server to synthesize proxy samples, eliminating the need to share private client data profiles.

This conceptual shift directly addresses the taxonomy question regarding whether KD constitutes a pure model compression technique. Rather than reducing the structural density of an existing topology, contemporary distributed KD acts as an architectural enabler: it allows highly compressed, edge-compatible student networks to converge under non-IID data distributions by anchoring their optimization trajectory to generalized, server-side structural insights.

III.1.2 Recent Advances and Trade-offs in Federated Quantization

While the foundational mechanisms of post-training and quantization-aware optimization are well-established, recent state-of-the-art research has shifted toward evaluating the specific systemic trade-offs that emerge when quantization is scaled within federated edge ecosystems [5]. Recent literature has extensively matted the operational updates provided by researchers, evaluating how lower numerical precision interacts with decentralized optimization trajectories.

The primary strengths identified in contemporary federated quantization research focus on multi-resource optimization:

- **Asymmetric quantization scheduling:** which allows the server to dynamically adjust bitwidth resolutions based on the instantaneous battery levels or wireless signal strengths reported by the edge nodes;
- **Gradient-specific compression:** which decouples the precision of forward weights from the backward updates, allowing clients to compress exclusively the uplink communication payload without compromising local model stability;
- **Mixed-precision hardware acceleration:** which enables heterogeneous edge devices to execute layer-specific quantization, assigning higher bitwidths to critical feature extraction layers and lower precision to redundant blocks.

Despite these structural advantages, the integration of low-precision arithmetic within federated loops introduces severe mathematical and operational vulnerabilities that represent open challenges in the current literature:

- **Gradient quantization noise aggregation:** which occurs when the server aggregates multiple low-precision client updates, causing the cumulative quantization errors to destabilize the global convergence path under heavily non-IID data distributions;
- **Representation alignment mismatches:** which arise when client architectures feature varying hardware acceleration units, leading to non-uniform clipping bounds that distort the global model’s decision boundaries during parameter averaging;
- **Precision-induced accuracy degradation in compact topologies:** which highlights how heavily compressed networks, such as shallow convolutional layers, exhibit high sensitivity to numerical truncation, causing catastrophic forgetting of minority class features.

This dual nature underscores that quantization cannot be evaluated as an isolated compression mechanism within distributed networks. Instead, contemporary studies emphasize the necessity of combining quantization with robust aggregation strategies, ensuring that the communication gains achieved by bitwidth reduction are not nullified by optimization divergence at the server side.

To address these fundamental limitations, recent breakthroughs in distributed optimization have moved beyond uniform bitwidth allocation, introducing specialized quantization methodologies that adapt to the statistical properties of deep networks [5]:

- **Vector quantization through clustering techniques:** which instead of rounding individual scalar weights independently, applies vector quantization algorithms like k -Means to group continuous parameter tensors into discrete clusters, replacing the original weight matrices with low-bitwidth cluster indices and a shared codebook of centroids [22];
- **Adaptive non-uniform quantization:** which discards linear quantization steps in favor of logarithmic or non-linear scaling functions that allocate higher numerical resolution to smaller weight values clustered near zero, thereby minimizing the distortion of critical gradient trajectories without increasing the overall bitwidth payload [6].

III.1.3 Pruning Strategies for Heterogeneous Architecture Allocation

Within distributed environments, pruning has evolved from a centralized network-trimming subroutine into an active orchestration mechanism designed to enforce structural heterogeneity across hardware tiers. Rather than deploying an identical, static model to all participating edge nodes, contemporary federated pruning methodologies dynamically slice the global network layers to fit the instantaneous storage and computational budgets of the local clients [8, 20].

To optimize sub-network extraction under volatile deployment conditions, the state of the art has converged toward three dominant architectural frameworks:

- **Channel-width scaling via HeteroFL:** involving the static partitioning of the global model into deterministic sub-networks where clients are assigned specialized profiles with horizontally reduced channel widths, allowing the central server to aggregate overlapping parameter indices without demanding architectural homogeneity [20];
- **Hierarchical ordered dropout via FjORD:** involving a dynamic, client-side execution protocol where parameters are sorted by structural

importance and nested hierarchically, enabling resource-constrained devices to evaluate their current battery and thermal states to adaptively prune the model down to the maximum executable width at every single round [8];

- **Data-driven importance pruning:** involving decentralized frameworks that go beyond pure hardware constraints by computing local Taylor expansions or magnitude-based saliency scores directly on the client’s private data, cutting out the least informative filters locally to compress the uplink transmission while preserving personalized feature representations [23].

To fully evaluate the impact of these strategies, it is necessary to examine the underlying algorithmic formulations that govern parameter selection and extraction during the distributed training loop.

In the channel-width scaling paradigm proposed by HeteroFL, the structural reduction is achieved by applying a deterministic scaling factor to the hidden dimensions of each layer. For a convolutional layer with a global channel configuration denoted as C_{global} , the framework defines a set of discrete complexity tiers $p \in \{1, \dots, P\}$, such that the local channel width is governed by a static reduction ratio [20]:

$$C_{local}^{(p)} = \lfloor \gamma_p \cdot C_{global} \rfloor \quad (\text{III.1.1})$$

where $\gamma_p \in (0, 1]$ represents the specific compression coefficient allocated to hardware tier p [20]. During the local forward and backward passes, the client updates exclusively the sub-matrix bounded by indices $[1 \dots C_{local}^{(p)}]$, effectively running a structured column-row pruning that shrinks both the computation on the edge device and the subsequent communication payload uploaded to the server [20].

Conversely, the FjORD framework introduces fluidity into this selection process via Ordered Dropout, transforming pruning from a rigid hardware categorization into an instantaneous state-driven decision. Mathematically, Ordered Dropout orders the hidden units or filters by structural priority, ensuring that lower-indexed parameters capture foundational features while higher-indexed blocks capture residual variations. Given a discrete space of candidate dropout rates $\mathcal{P}$, each client evaluates its instantaneous local resource availability—such as available memory, processing throughput, or volatile energy budgets—at the beginning of each communication round.

Based on these hardware constraints, the node dynamically selects the maximum sustainable dropout rate $p_{max} \in \mathcal{P}$ that the edge hardware can safely compute without incurring resource exhaustion [8].

The client then extracts a self-contained, nested sub-network by preserving exclusively the parameters associated with the index range up to $\lceil p \cdot K_l \rceil$, where K_l denotes the original layer width. Because the sub-networks are nested hierarchically, the foundational leftmost layers are updated by all participating nodes, creating an inherent regularization effect that stabilizes the global model boundaries even when aggregated across highly volatile, asymmetric client dropouts [8].

Finally, data-driven importance pruning approaches shift the pruning criteria from hardware availability to statistical data significance. Instead of relying on a deterministic geometric cutting like Ordered Dropout, these methodologies force each client to compute a localized saliency score S for every filter or weight component w . This score is typically formalized using a first-order Taylor expansion of the local loss function $\mathcal{L}$, which estimates the prediction error incurred if that specific parameter were zeroed out [5]:

$$S(w) = \left| \frac{\partial \mathcal{L}}{\partial w} \cdot w \right| \quad (\text{III.1.2})$$

The client then generates a personalized sparse mask by sorting the parameters based on $S(w)$ and removing those below a non-uniform threshold, drastically reducing the non-zero weight count before the aggregation phase. While this approach maximizes task accuracy on highly non-IID local datasets, it shifts a severe computational burden onto the client, as computing the first-order gradients for saliency evaluation requires additional forward-backward iterations that can deplete the resources of memory-constrained microcontrollers [23].

IV Contribution

IV.1 Framework Overview

This work introduces *FedComp*¹, a modular and extensible experimental framework designed for the systematic evaluation and empirical profiling of model compression techniques within Federated Learning (FL) environments. Built on top of *Flower* [13], one of the most widely adopted environments for federated learning research, *FedComp* acts as a specialized experimental testbed. Its primary objective consists of the design, architectural definition, and software development of an infrastructure aimed at analyzing the multi-dimensional trade-offs between predictive performance (accuracy and loss) and on-device computational efficiency.

The framework is specifically conceived to address the severe operational constraints typical of Edge AI and TinyML resource-constrained nodes, where memory footprints, processing capabilities, and network bandwidth are strictly bounded. By providing a standardized and isolated testing environment, *FedComp* enables researchers to profile algorithmic compression mechanisms under controlled conditions, establishing clear performance baselines and optimization boundaries.

IV.2 System Architecture and General Workflow

To ensure high modularity, extensibility, and statistical isolation of the experimental variables, the framework has been organized into four independent yet conceptually aligned sub-projects. Each sub-project is encapsulated within a dedicated directory containing its own isolated implementation of the *Flower-PyTorch* pipeline:

- **Baseline Standard FL:** a native configuration based on the standard

¹<https://github.com/VanniMaceria/tesi-magistrale>

Flower protocol and a full-precision model architecture, lacking any algorithmic compression. This module serves as the primary term of comparison to quantify the performance degradation or optimization margins introduced by the other techniques;

- **Federated Knowledge Distillation (FKD):** a specialized component implementing an inverted distillation paradigm tailored to overcome edge hardware limitations. In this configuration, the local client models act as distributed *Teachers*, capturing fine-grained features from their private data partitions. Conversely, the central server trains a global *Student* model, which is designed with a highly compact architecture. Instead of performing standard parameter averaging, the server uses the collective knowledge emitted by the client teachers to train this global student;
- **Ordered Dropout (OD):** a specialized component designed to mitigate hardware heterogeneity across edge clients, heavily inspired by the architectural paradigms introduced in the *FjORD* framework [8]. Instead of treating network pruning as a post-training compression technique, this module embeds a dynamic *width slicing* mechanism directly into the network’s forward pass using functional operators. The network architecture is made elastic by sub-sampling the weight matrices in real time based on an ordered dropout scale p . This process structurally forces the global model to pack the most critical latent features within the top-left sub-matrices, which are consistently updated by all participating nodes, while peripheral sub-structures are trained only by clients with higher computational capacities. In this implementation, a static profiling strategy is deployed through a deterministic identifier mapping, binding each client to a fixed capacity tier ($p \in \{0.25, 0.5, 0.75, 1.0\}$) to simulate a realistic, non-uniform IoT network topology;
- **Quantization-Aware Training (QAT):** a specialized component designed to evaluate the model’s resilience to low-precision hardware constraints within a federated environment. By integrating Fake Quantization operators, this module simulates the deterministic rounding errors, truncation effects, and limited dynamic range typical of 8-bit integers (*int8*) directly during the forward pass. Conversely, the backward pass bypasses the non-differentiable step-like nature of quantization

by maintaining high-precision *float32* gradients, leveraging a Straight-Through Estimator (STE) to ensure numerical stability and prevent gradient vanishing. This configuration allows the model to learn to actively compensate for quantization noise during the cooperative optimization loop, while keeping full-precision latent weights on the server for fine-grained updates.

IV.2.1 Shared Execution Backbone and Standardized Profiling

Crucially, all four sub-projects share an identical underlying logical pipeline concerning deterministic seeding, data ingestion, and both IID and Non-IID partitioning using several datasets. Furthermore, they inherit a unified, analytically-driven resource profiling module to track on-device floating-point operations (FLOPs), estimated energy consumption in Joules, and network bandwidth overhead in Megabytes. This strict structural standardization guarantees that any observed performance shift is purely attributable to the core algorithmic differences of the compression techniques.

IV.2.2 General Federated Workflow

Across all implemented sub-projects, the execution lifecycle follows a synchronized, round-based federated orchestration model structured into three operational phases:

1. **Downlink Distribution:** the central server orchestrator packages the current global model state alongside runtime hyper-parameters (e.g., learning rate) into a network payload and broadcasts it to the participating subset of edge nodes;
2. **Local Edge Execution and Profiling:** each selected client loads its private data partition slice, updates the received model via local Stochastic Gradient Descent (SGD) iterations, and validates the updated state. Immediately after training, the client executes the analytical profiling engine to measure its local resource footprint (bandwidth, energy, FLOPs) before returning the updated parameters and telemetry metrics back to the server;
3. **Uplink Aggregation and Centralized Evaluation:** the server collects the client response payloads, aggregates the incoming updates accord-

ing to the specific technique’s logic, records the telemetry metrics using scientific notation, and performs a global validation check to monitor overall system convergence.

IV.3 An in-depth look at the components

What follows is an in-depth examination of the components, covering both their static aspects (behavior in isolation) and dynamic aspects (behavior when communicating with other components).

IV.3.1 Baseline Standard FL

The baseline module establishes a native, uncompressed federated learning pipeline designed to assess the full-precision behavior of neural network structures across distributed environments. This setup isolates three independent operational components, each executing distinct functional behaviors:

Shared Core Compute Engine (task.py)

The shared core compute engine operates as the underlying algorithmic backbone of the entire architecture, providing the fundamental mathematical primitives, data transformation pipelines, and network modeling definitions required to maintain structural alignment across both centralized and distributed nodes. Statistically, this module defines the `Net` architecture, a full-precision Convolutional Neural Network (CNN) structured with sequential convolutional layers, spatial max-pooling operators, and linear dense layers that map inputs to an output space dynamically scaled to match the classification requirements of the targeted dataset. Beyond model definition, this component is responsible for enforcing complete numerical reproducibility across parallel client simulations. It encapsulates a deterministic seeding mechanism that overrides non-deterministic scheduling variations within the computing backend and random number generators. Furthermore, it manages the dataset ingestion and transformation layer, handling structural tensor normalizations, generic key remapping, and the partition splitting boundaries that feed raw inputs uniformly into the localized client loops. Lastly, this module hosts the analytical evaluation engine responsible for benchmarking the computational complexity of the network layout before deployment. By instantiating a localized profiling routine, the engine injects

a synthetically structured tensor matching the operational input shape of the framework. It monitors the layer-by-layer execution path to calculate the precise count of total network parameters and Multiply-Accumulate (MAC) operations under a verbose-masked runtime environment. Since a standard MAC operation encapsulates both a multiplication and an addition, the engine applies a deterministic scaling factor to convert these actions into raw Floating-Point Operations (FLOPs), outputting the exact analytical complexity required by the client emulators to anchor their physical energy and runtime resource projections.

Client-Side Edge Emulator (client_app.py)

The client-side edge emulator encapsulates the computational and analytical execution loop restricted to individual distributed edge nodes. Upon instantiation, each independent client instance ingests a private data partition uniquely bound to its node identifier. Rather than operating as a passive training worker, the emulator hosts localized evaluation routines by dividing its assigned partition into training and validation subsets guided by an incoming round-specific configuration seed. Once local stochastic gradient descent iterations are concluded, the client node executes a static analytical profiling engine. This component isolates the model’s structure to mathematically estimate the local IoT deployment footprint, calculating the exact network bandwidth required for bi-directional weight exchanges and the cumulative runtime energy consumption over the trained epochs before wrapping these values into a metadata payload. To mathematically quantify this local deployment footprint, the analytical profiling engine inside each client translates the structural properties of the network into tangible edge resource metrics. First, the bi-directional network bandwidth overhead (B_{MB}), measured in Megabytes, is calculated by evaluating the total parameter count (P) of the model full-precision weights as follows:

$$B_{MB} = \frac{P \times 4 \times 2}{1024 \times 1024} \quad (\text{IV.3.1})$$

where the factor of 4 represents the byte size of a standard float32 primitive, and the multiplier of 2 accounts for the full symmetric communication overhead (both uploading local updates and downloading the new global state).

Concurrently, the total computational complexity of the local training phase is estimated. The framework approximates the total training operations

($\text{FLOPs}_{\text{train}}$) performed by the node during the current round by anchoring them to the single-image inference complexity ($\text{FLOPs}_{\text{inf}}$) using the following formulation:

$$\text{FLOPs}_{\text{train}} = \text{FLOPs}_{\text{inf}} \times 3 \times N_{\text{samples}} \times E_{\text{local}} \quad (\text{IV.3.2})$$

Here, N_{samples} denotes the exact number of local training samples available in the client’s partition, E_{local} is the number of local epochs, and the factor of 3 serves as the literature standard approximation to model the computational overhead of backpropagation (where the forward pass accounts for $1 \times \text{FLOPs}$ and the backward pass takes approximately $2 \times \text{FLOPs}$).

Finally, these two components are linearly combined to estimate the overall energy consumption (E_{total}), measured in Joules:

$$E_{\text{total}} = (\text{FLOPs}_{\text{train}} \times \kappa_{\text{computation}}) + (B_{\text{MB}} \times \kappa_{\text{network}}) \quad (\text{IV.3.3})$$

Within this formula, the framework applies two physical optimization constants derived from hardware literature: $\kappa_{\text{computation}} = 10^{-11}$ J/FLOP models the energy cost of a standard math operation accounting for local cache SRAM access overhead on commercial TinyML devices [15], while $\kappa_{\text{network}} = 0.05$ J/MB models the typical power consumption of a standard Wi-Fi wireless transmission module [16].

Server-Side Orchestrator (server_app.py)

The server-side orchestrator acts as the centralized global state machine responsible for coordinating the synchronized progression of the federated framework. At its core, it initializes the global model parameters in a standard high-precision format (float32) and provisions a centralized validation pipeline fed by a dedicated global dataset split to independently benchmark the system’s generalization capabilities. During execution, the server operates as a receiving hub that linearly averages the incoming model weights using *FedAvg* aggregation strategy. Simultaneously, the orchestrator handles the aggregation of the heterogeneous resource payloads emitted by the edge nodes, computing global means and executing floating-point safe logging routines in scientific notation to ensure data integrity across long training horizons.

Component Collaboration and Dynamic Workflow

The orchestration of a standard federated learning communication round relies on a continuous, synchronized exchange of message envelopes between the centralized server and the distributed clients. Instead of executing as a single monolithic process, the framework structures this operational interaction across three sequential phases:

1. Server Initialization and Downlink Broadcasting

The round begins under the coordination of the server-side orchestrator. The server initializes the step by packaging the current weights of the global model along with the runtime configurations (such as the targeted learning rate) into an outbound message packet. Through the underlying simulation engine, this network envelope is then broadcast to the selected subset of active virtual edge nodes;

2. Local Edge Training, Evaluation, and Profiling

Upon receiving the server envelope, operational control shifts to the client-side edge emulators. This phase executes through the following steps:

- *Data Preparation*: each selected client unpacks the incoming global weights, loads its private data partition slice, and interfaces with the shared core compute engine (`task.py`) to isolate its training and validation boundaries;
- *Stochastic Optimization*: once local data structures are ready, the client updates the model by executing the local training loop via Stochastic Gradient Descent (SGD) over the specified local epochs;
- *Local Validation*: immediately following the optimization phase, the client evaluates the fresh weights on its local validation subset to record the post-training accuracy and loss scores;
- *Resource Profiling & Uplink Packaging*: before concluding its execution, the client emulator invokes the analytical profiling logic to quantify its edge resource footprint, as described in Section Client-Side Edge Emulator. All computed resource metrics, local evaluation scores, and newly updated weights are wrapped into a single response envelope and transmitted back to the server.

3. Server Aggregation, Logging, and Centralized Evaluation

The cycle closes as the server-side orchestrator regains control:

- *Federated Aggregation*: the server intercepts all incoming client responses, extracting the locally updated weights to compute the new global model state via a linear Federated Averaging (*FedAvg*) routine;
- *Efficiency Logging*: simultaneously, it parses the attached resource payloads to compute global efficiency averages. To prevent precision loss and formatting corruption across long execution runs, the server logs these aggregated metrics to a structured file using scientific notation;
- *Global Convergence Check*: finally, the orchestrator passes the new global model back to the data engine to run a centralized validation check against a dedicated proxy dataset, verifying the overall convergence quality of the system before advancing to the next communication round.

IV.3.2 Federated Knowledge Distillation

To mitigate the communication and memory bottlenecks typical of edge deployments, the baseline architecture is extended by integrating a Federated Knowledge Distillation (FKD) paradigm. Rather than averaging parameter tensors directly as in traditional Federated Averaging, this approach uses the clients' updated models as an ensemble of "Teachers" to train a compact global "Student" model on the server side via soft-label distillation.

Shared Core Compute Engine (task.py)

The dataset pipeline and client-side partitioning logic remain fully consistent with the baseline architecture described previously, ensuring a fair experimental comparison. However, two primary modifications are introduced at this layer:

- **Compressed Model Architecture (TinyNetIoT)**: the neural network backbone is replaced with a highly lightweight model optimized for resource-constrained edge devices. Compared to the baseline model, the channel capacity of the convolutional layers is halved (reducing `conv1` output channels from 6 to 3, and `conv2` from 16 to 8). Consequently, the fully connected layers are scaled down to accept 128 input features, substantially reducing total parameter count and floating-point operations (FLOPs) per inference;

- **Server Proxy Dataset Provider:** to perform server-side distillation without violating data privacy, the module introduces `load_proxy_dataset()`. This function extracts a small, non-overlapping dataset slice (first 500 samples) from the global dataset split. This proxy dataset acts as an unlabeled transfer set on the server to elicit prediction logits from the models.

Distillation Core (distiller.py)

This module defines the mathematical objective for transferring knowledge from the client ensemble to the global student model.

- **Temperature-Scaled Logit Softening:** instead of comparing hard class labels, predictions from both Student (y_{student}) and Teachers (y_{teacher}) are softened using a temperature parameter $T = 3.0$:

$$p_i = \frac{\exp(y_i/T)}{\sum_j \exp(y_j/T)} \quad (\text{IV.3.4})$$

High values of T produce a smoother probability distribution, forcing the Student to learn the dark knowledge, the subtle spatial geometric correlations between non-predicted classes embedded within the Teacher ensemble;

- **Distillation Loss Formulation:** the objective function is defined as the Kullback-Leibler (KL) Divergence between the log-probabilities of the Student and the averaged probability predictions of all active Teachers, scaled by T^2 to keep gradient magnitudes consistent across temperature variations:

$$\mathcal{L}_{\text{KD}} = T^2 \times \text{KL} \left(\log_{\text{softmax}} \left(\frac{y_{\text{student}}}{T} \right) \parallel \text{mean} \left(\text{softmax} \left(\frac{y_{\text{teacher}}}{T} \right) \right) \right) \quad (\text{IV.3.5})$$

Optimization is carried out using the Adam optimizer over the proxy dataset batches.

Server-Side Orchestration (server_app.py)

The central server replaces the native FedAvg logic with a custom orchestration class, `DistillationStrategy`:

- **Teacher Ensemble Reconstruction:** upon receiving response packets from participating nodes, the server deserializes the weight byte-

streams into individual `TinyNetIoT` instances, setting them to evaluation mode (`eval()`) to act as frozen Teachers;

- **Student Training Loop:** a fresh `TinyNetIoT` instance is instantiated as the global Student. The server executes `train_distillation()`, passing the proxy dataset, the frozen Teachers, and the Student. The Student model learns from the average ensemble logits and becomes the updated global model state returned for the next round;
- **Metrics Aggregation:** logistical and IoT profiling metrics (bandwidth, FLOPs, and energy) sent by the clients are aggregated identically to the baseline pipeline, ensuring consistent scientific logging.

Client-Side Edge Emulator (`client_app.py`)

The operational loop on the client remains functionally identical to the baseline emulator, with two adjustments:

- The client instantiates the compressed `TinyNetIoT` backbone instead of the larger baseline model;
- Because `TinyNetIoT` has significantly fewer parameters (P) and lower single-inference complexity ($\text{FLOPs}_{\text{inf}}$), the analytical profiling engine inside the client automatically evaluates a drastically reduced network bandwidth footprint (B_{MB}) and local training energy cost (E_{total}).

Component Collaboration and Dynamic Workflow

The operational lifecycle of a distillation-based communication round progresses through three coordinated phases:

1. **Downlink Distribution Phase:** the server-side orchestrator broadcasts the current global `TinyNetIoT` (Student) parameter weights and runtime hyperparameters to the sampled client subset;
2. **Local Edge Training and Profiling Phase:** each selected client node receives the parameters, loads its private data partition via `task.py`, and executes local training epochs using stochastic gradient descent. Post-training evaluation accuracy and loss are computed locally. Subsequently, the client invokes the analytical profiling logic to compute its reduced bandwidth and energy metrics based on `TinyNetIoT`'s

structural properties. The updated local weights, metrics, and sample counts are packaged into a response envelope and sent back to the server;

3. **Server-Side Distillation Phase:** instead of executing a weighted linear parameter average, the server's `DistillationStrategy` intercepts the incoming client weights and reconstructs them into an ensemble of functional Teacher models. The server then loads the 500-sample proxy dataset and trains a fresh `TinyNetIoT Student` model using temperature-scaled KL-Divergence loss against the average predictions of the Teachers. Once server-side distillation completes, the new Student parameters are serialized and evaluated against the centralized dataset to monitor convergence quality before initiating the subsequent round.

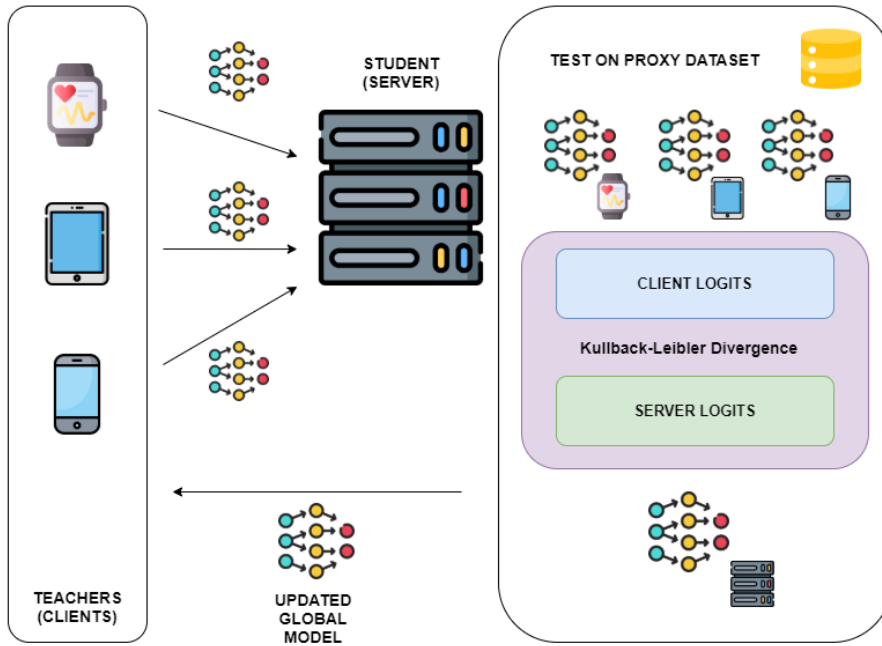


Figure IV.1: Workflow of Federated Knowledge Distillation

IV.3.3 Ordered Dropout

To address the hardware heterogeneity of edge environments without deploying separate models, the baseline architecture is extended using the Ordered Dropout (OD) framework. By enforcing a nested representation (a "Matryoshka" structure) within a single global model, OD allows client

nodes to train sub-networks of varying capacities ($p \in \{0.25, 0.5, 0.75, 1.0\}$) proportional to their computational and energy constraints.

Shared Core Compute Engine and Elastic Width Backbone (task.py)

To accommodate hardware heterogeneity across edge devices without requiring multiple distinct architectures, the core neural network (`Net`) is redesigned to support dynamic width elasticity at runtime. Rather than operating with a fixed capacity, the backbone enables instant execution at fractional sub-network widths $p \in \{0.25, 0.5, 0.75, 1.0\}$, effectively embedding smaller models within the original baseline architecture.

The fundamental mechanism driving this architecture is the implicit enforcement of an ordered representation across the parameter space. Crucially, this ordering is not achieved via explicit sorting or post-processing algorithms; rather, it emerges as a direct mathematical consequence of gradient dynamics during local training. Because weight matrices are systematically sliced starting from the origin (the top-left sub-region, indexed from 0 up to the active boundary), the parameters residing in this core region are activated and updated by *all* participating client nodes, regardless of their assigned p value. Consequently, these primary channels receive the highest volume of gradient updates across the federated network, forcing them to learn the most essential, coarse-grained visual features (e.g., primary edges and fundamental shapes). In contrast, peripheral channels located towards the bottom and right edges of the weight matrices are updated exclusively by high-capacity nodes ($p = 1.0$). They receive significantly fewer gradient updates, causing them to capture secondary refinements and detailed representations. During the forward pass, the network dynamically determines the active width of each layer as a function of the target fraction p . By selecting these top-left sub-regions, the network scales both its spatial convolutional representations and its fully connected hidden layers proportionally. The output classification layer maintains a fixed dimension corresponding to the full dataset targets while seamlessly adjusting its input feature receptive field to match the active preceding layer.

To emulate physical resource constraints across the federated ecosystem, client nodes are deterministically assigned a fixed operational profile p derived from their unique node identifiers. Furthermore, because standard profiling frameworks struggle to inspect dynamic, functionally sliced architectures, the module integrates an analytical resource estimation engine. This

engine evaluates parameter footprints and floating-point operations directly from the active channel boundaries, providing accurate telemetry on computational complexity and network bandwidth for any given sub-network fraction.

Server-Side Multi-Profile Evaluator (server_app.py)

While global parameter aggregation utilizes standard *FedAvg*, the orchestration loop introduces significant modifications in the evaluation pipeline:

- **Multi-Profile Centralized Evaluation (`global_evaluate`):** at the end of each communication round, the full global model is not evaluated solely at full capacity. Instead, the server iterates across all discrete width profiles ($p \in \{0.25, 0.5, 0.75, 1.0\}$) on the centralized test dataset. This monitors the convergence and accuracy curve of both the pruned sub-networks and the complete model simultaneously;
- **Extended Telemetry Reporting:** the aggregation logic logs the assigned p profile alongside accuracy, loss, bandwidth, and energy metrics to individual client records, allowing performance tracking across distinct sub-network capacities.

Client-Side Heterogeneous Node Emulator (client_app.py)

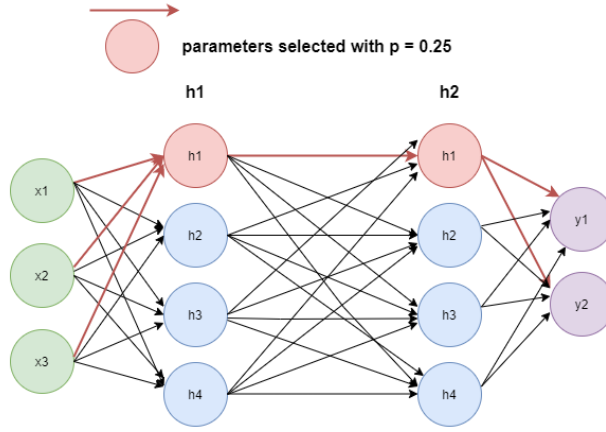
The client execution loop adapts dynamically to the profile assigned to the node:

- **Profile Retrieval:** upon activation, the client evaluates its local capacity p via `get_p_from_id(partition_id)`;
- **Sub-Network Local Training:** during local training and validation, the client passes p to `train_fn()` and `test_fn()`, ensuring gradients update only the top-left sub-matrices corresponding to p ;
- **Proportional Resource Accounting:** network bandwidth and training energy are evaluated using the analytical $P(p)$ and $\text{FLOPs}(p)$ metrics. Nodes operating with $p = 0.25$ report significantly reduced transmission volumes and computational energy compared to full-capacity nodes ($p = 1.0$).

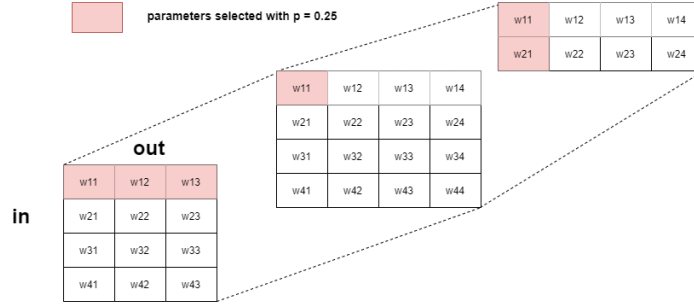
Component Collaboration and Dynamic Workflow

The operational lifecycle for an Ordered Dropout communication round proceeds through three phases:

1. **Downlink Broadcast Phase:** the central server serializes the global parameter set representing the full-capacity network ($p = 1.0$) and distributes it to the sampled client subset;
2. **Profile-Guided Local Execution Phase:** each client receives the full parameter state but determines its assigned capacity p based on its node index. During local training, the forward and backward passes execute exclusively on the sliced weight sub-matrices specified by p . Consequently, parameters in the top-left corner receive frequent gradient updates from all clients, whereas peripheral parameters receive updates only from clients with higher p values. Upon training completion, the client evaluates its reduced resource footprint analytically and returns its updated parameters and telemetry;
3. **Aggregation and Multi-Profile Evaluation Phase:** the server aggregates the incoming model updates using *FedAvg*. Once the global model parameters are updated, `global_evaluate()` evaluates the new state across all four sub-network profiles ($p = 0.25, 0.5, 0.75, 1.0$), logging the multi-tier performance metrics before initiating the next round.



(a) Graph representation



(b) Matrix slicing representation

Figure IV.2: Structural representation of the network and parameter slicing under profile $p = 0.25$.

IV.3.4 Quantization-Aware Training (QAT) Extension

To enable execution on edge microcontrollers and reduce network transmission overhead, the baseline framework is extended with Quantization-Aware Training (QAT). Standard neural network training relies on 32-bit floating-point (`float32`) representations. However, edge hardware often lacks specialized Floating Point Units (FPUs) and operates far more efficiently on 8-bit integers (`int8`).

Directly quantizing parameters post-training (Post-Training Quantization) often induces severe degradation due to truncation and clipping errors. QAT addresses this by modeling 8-bit quantization noise directly during local gradient optimization, allowing the network parameters to adapt to low-precision representation errors before deployment.

Shared Core Compute Engine and Quantization Stubs (task.py)

Unlike the standard baseline network, the core model architecture (Net) is modified to incorporate explicit quantization boundaries:

- **Quantization Stubs (QuantStub and DeQuantStub):** the network architecture introduces a `QuantStub` entry module and a `DeQuantStub` exit module. The input stub simulates sensor-level analog-to-digital conversion by converting continuous input signals into 8-bit integer domain bounds. The output stub converts low-precision internal representations back to `float32` space, enabling loss computation and objective evaluation;
- **Fake Quantization Operations:** during the forward pass, internal layer weights and activation maps are mathematically constrained (clipping to $[-128, 127]$ and rounding). This simulates the exact precision loss of target microcontroller engines (such as ARM QNNPACK) while keeping the underlying parameters in floating-point format to prevent gradient vanishing during backpropagation.

Model Expansion and QAT Configuration (server_app.py)

To manage serialized parameter states across the federated server-client architecture, the server execution module introduces specific structural preparations:

- **Topology Expansion (prepare_model_for_qat):** before parameters are serialized into `ArrayRecord` buffers or passed to `FedAvg`, the model structure is explicitly transformed using PyTorch's QAT preparation logic configured for the `qnnpack` engine. This attaches scale and zero-point quantization observers directly to layer modules;
- **Latent Floating-Point Aggregation:** global aggregation on the server operates over full-precision "latent" weights. This ensures that parameter updates received from clients are aggregated without compounding truncation errors at each round.

Local Low-Precision Trainer (client_app.py)

The client execution loop adapts the baseline training process to handle low-precision forward passes alongside full-precision backward passes:

- **Structural Alignment Prior to Deserialization:** upon receiving parameter arrays from the server, the client must invoke `prepare_local_model_for_qat()` on its locally instantiated network *before* executing `load_state_dict()`. This guarantees structural state-dict parity between the server’s observer-augmented architecture and the client instance;
- **Latent Weight Backpropagation:** during local training epochs, forward passes suffer quantization noise, whereas gradient updates are applied to underlying full-precision latent weights;
- **Reduced Footprint Telemetry:** bandwidth metrics are calculated assuming an 8-bit representation (1 byte per weight parameter rather than 4 bytes), reflecting the compressed communication burden achieved in low-precision deployment scenarios.

Component Collaboration and Dynamic Workflow

The operational lifecycle for a QAT-enabled federated communication round proceeds as follows:

1. **Server Initialization and Downlink Transmission:** the central server instantiates the baseline backbone, transforms its structure via `prepare_model_for_qat()`, and exports the full state dictionary (comprising latent weights and quantization observer states). The parameter record is then broadcast to selected clients;
2. **Client Local Training with Fake Quantization:** each participating client prepares its local model instance with identical QAT configurations before loading the server’s state dictionary. During training:
 - The input tensor x passes through `QuantStub`, entering the simulated `int8` domain;
 - Intermediate activations and weight layers apply fake quantization operators (clipping and rounding);
 - `DeQuantStub` maps the final predictions back to floating-point representation to calculate `CrossEntropyLoss`;
 - Gradients update the latent `float32` weights, adapting them to tolerate lower precision.

3. **Uplink Transmission and Global Aggregation:** the client returns its updated latent weights and scale parameters to the central server. The server aggregates the incoming client states using standard FedAvg, yielding an updated global state ready for the subsequent round or final post-training conversion.

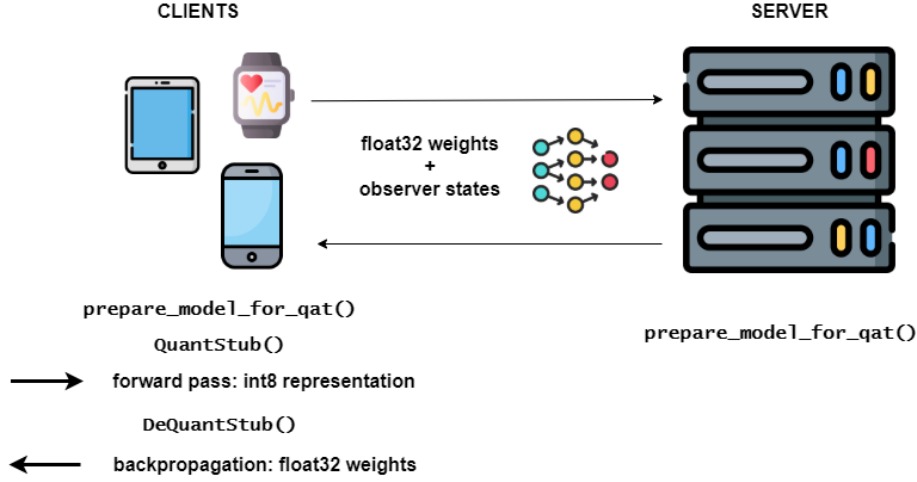


Figure IV.3: Workflow of Federated Quantization Aware Training

IV.4 Scientific and Technical Contributions

The primary goal of this work is to bridge the gap between algorithmic compression theory and practical edge resource profiling in Federated Learning. The main contributions of the *FedComp* framework to the research community can be summarized as follows:

- **A Standardized and Isolated Benchmarking Infrastructure:** the framework provides a clean, modular repository architecture where four major compression paradigms (Baseline, FKD, OD, and QAT) operate under strictly aligned data partitioning, deterministic seeding, and network conditions. This eliminates confounding variables and allows researchers to isolate pure algorithmic trade-offs;
- **Integrated Multi-Dimensional Resource Profiling:** unlike conventional FL simulators that focus solely on top-1 accuracy and loss, *FedComp* embeds a static, analytical profiling engine into the client run-time loop. This enables real-time tracking of Floating-Point Operations

(FLOPs), memory footprint scaling, bi-directional network bandwidth overhead (MB), and estimated edge energy consumption (J);

- **Empirical Validation of Low-Precision and Elastic Heterogeneity:** the implementation provides an extensible testbed to evaluate how modern compression techniques—such as sub-network width-slicing via Ordered Dropout and non-differentiable 8-bit integer simulation via Quantization-Aware Training—behave under cooperative optimization and non-IID data distributions;
- **Open-Source and Reproducible Edge AI Research Tool:** to foster reproducibility and support future research in TinyML and Federated Learning, the entire software architecture, including dataset partitioners and profiling modules, is made publicly available as an open-source codebase.

V Experimentation

This chapter presents the empirical evaluation framework designed to comprehensively analyze and compare the performance, communication overhead, and resource efficiency of the model compression strategies integrated within the federated pipeline. To rigorously evaluate the framework under varying degrees of task complexity and statistical heterogeneity, a diverse suite of benchmark datasets and data distribution regimes is adopted.

V.1 Experimental Datasets and Partitioning Regimes

To validate the robustness and adaptability of the proposed federated framework, encompassing Baseline Standard FL, Federated Knowledge Distillation (FKD), Ordered Dropout, and Quantization-Aware Training (QAT), experiments are conducted using three distinct image classification benchmarks. These datasets scale in visual complexity and naturally exhibit the structural characteristics required to simulate real-world IoT environments.

V.1.1 Benchmark Datasets

1. **MNIST (Modified National Institute of Standards and Technology):** as a foundational baseline in computer vision and deep learning, the MNIST dataset comprises 60,000 training images and 10,000 testing images of grayscale handwritten digits (28×28 pixels) spanning ten classes (0 through 9). Due to its low spatial complexity and clear feature distribution, MNIST serves as the primary sanity check to verify baseline model convergence and basic optimization stability across all compression variants [17];
2. **Fashion-MNIST:** to bridge the gap between simplistic digit recognition and real-world visual tasks, Fashion-MNIST is introduced as a direct

drop-in replacement for standard MNIST. It shares an identical image dimension (28×28 grayscale pixels), data volume (60,000 training and 10,000 testing samples), and class count (10 categories of clothing items). However, Fashion-MNIST presents significantly higher textural complexity, edge variance, and overlapping feature distributions, making it a robust testbed for evaluating the generalization capabilities of compressed convolutional architectures under resource constraints [18];

3. **FEMNIST (Federated Extended MNIST):** to accurately simulate real-world Cross-Device Federated Learning and the inherent statistical challenges of Internet of Things (IoT) ecosystems, the FEMNIST dataset is utilized. Derived from the LEAF benchmark suite, FEMNIST is a naturally partitioned dataset built by collecting handwritten digits and characters from a vast population of distinct writers. Just like the two previous datasets, the samples have dimensions of 28×28 grayscale pixels; however, the dataset contains a single test split of 814,277 rows and comprises 62 classes [13, 19]. It inherently embodies extreme statistical heterogeneity, where each user (*writer_id*) contributes a localized data distribution that reflects their unique writing style, frequency of specific characters, and sample size imbalances.

V.1.2 Data Distribution Regimes: IID vs. Non-IID

To systematically evaluate the resilience of the framework against statistical heterogeneity, a primary challenge in federated edge training, the datasets are distributed across participating clients under two distinct operational regimes:

- **Independent and Identically Distributed (IID) Regime:** in this configuration, the global training dataset is shuffled uniformly and partitioned equally among all participating clients. Consequently, each edge node receives a local data distribution that closely mirrors the global population statistics, serving as an idealized reference scenario characterized by minimal client drift and optimal convergence dynamics;
- **Non-Independent and Identically Distributed (Non-IID) Regime:** to replicate realistic federated IoT environments, data are partitioned to introduce severe statistical skew. For benchmarks like MNIST and

Fashion-MNIST, an extreme non-IID partitioning strategy is enforced using a *Dirichlet* distribution with a concentration parameter of $\alpha = 0.1$, which heavily biases the label distribution across clients. Furthermore, for FEMNIST, the natural writer-based grouping inherently introduces feature distribution shifts and label imbalance.

Every optimization and compression strategy investigated in this work is rigorously benchmarked under both regimes to evaluate its respective robustness against statistical heterogeneity and client drift.

V.2 Experimental Configuration and Hyperparameters

To ensure complete reproducibility and fair benchmarking across all evaluation modules, the system is configured across distinct experimental profiles depending on the target dataset and data distribution regime.

V.2.1 Network Architectures

The experimental framework evaluates two tailored Convolutional Neural Network (CNN) topologies designed for image classification across MNIST, Fashion-MNIST, and FEMNIST.

- **Baseline CNN Architecture (Net):** the standard full-precision model features a sequential feature extraction pipeline inspired by LeNet architectures [3]. It ingests single-channel grayscale images ($1 \times 28 \times 28$) through a first convolutional layer (*Conv2d*) with 6 output channels and a 5×5 kernel, followed by a spatial max-pooling operator (2×2 , stride 2). A second convolutional layer maps 6 input channels to 16 output channels using a 5×5 kernel, succeeded by a second max-pooling layer. After spatial flattening (resulting in a $16 \times 4 \times 4$ feature tensor), the network routes representations through a sequence of fully connected dense blocks: an initial dense layer mapping 256 inputs to 120 neurons, a second hidden layer scaling down to 84 neurons, and a final classification layer projecting to the target output space (84 to 10 for MNIST and Fashion-MNIST and 84 to 62 classes for FEMNIST);
- **Compact Edge Architecture (TinyNetIoT):** specifically integrated within the Federated Knowledge Distillation (FKD) paradigm to emu-

late highly resource-constrained IoT endpoints, this compressed backbone halves the structural capacity of the baseline model. The initial convolutional layer reduces output channels from 6 to 3, while the second convolutional layer scales down from 16 to 8 filters ($3 \rightarrow 8$ channels) with a preserved 5×5 kernel size. Following spatial pooling, the flattened tensor ($8 \times 4 \times 4$) feeds into a leaner fully connected cascade consisting of a 128-to-60 hidden projection layer, a subsequent layer scaling from 60 to 40 neurons, and a final classification head mapping 40 features to the output classes. This structured halving substantially reduces parameter memory footprints and floating-point operations per inference.

V.2.2 Dataset-Specific Hyperparameter Profiles

The federated simulations maintain a fixed population of $K = 10$ active clients across all experiments, with an evaluation fraction of $\text{fraction-evaluate} = 0.5$ (meaning half of the clients test the model at each round). However, local epochs are set to $E_{\text{local}} = 1$ across all setups, while rounds, batch sizes, and learning rates are adjusted per dataset:

1. MNIST (IID and Non-IID Regimes):

- **Global Rounds (R):** 15 communication rounds;
- **Local Epochs (E_{local}):** 1 epoch per round;
- **Batch Size (B):** 32;
- **Learning Rate (η):** 0.01.

2. Fashion-MNIST (IID and Non-IID Regimes):

- **Global Rounds (R):** 15 communication rounds;
- **Local Epochs (E_{local}):** 1 epoch per round;
- **Batch Size (B):** 1;
- **Learning Rate (η):** 0.001.

3. FEMNIST (IID Partitioning):

- **Global Rounds (R):** 10 communication rounds (reduced to account for the heavy computational load of processing the large volume of local samples across the 10 clients);

- **Local Epochs (E_{local}):** 1 epoch per round;
- **Batch Size (B):** 1;
- **Learning Rate (η):** 0.001.

4. FEMNIST (Non-IID Partitioning):

- **Global Rounds (R):** 30 communication rounds (extended to ensure full model convergence under severe statistical heterogeneity);
- **Local Epochs (E_{local}):** 1 epoch per round;
- **Batch Size (B):** 1;
- **Learning Rate (η):** 0.001.

V.2.3 Specific Framework Parameters

- **Knowledge Distillation (FKD):** for the server-side distillation phase, the temperature is set to $T = 3.0$ within the Kullback-Leibler divergence loss function ($T^2 \times \text{KLDiv}$) to soften probability outputs. The global Student model is optimized over a proxy dataset comprising 500 unlabelled samples using the Adam optimizer with a learning rate of $\eta = 0.01$;
- **Ordered Dropout Profiles:** active sub-network capacities are evaluated across discrete width fractions $p \in \{0.25, 0.5, 0.75, 1.0\}$.

V.3 Resource Profiling and Telemetry Methodology

To evaluate the engineering trade-offs of model compression across resource-constrained environments, the experimental framework leverages the analytical profiling methodology established in Chapter IV. Rather than relying solely on empirical runtime measurements, the system tracks deterministic resource metrics during client execution, which are automatically exported into structured CSV telemetry logs categorized into two distinct types:

- **Client-Specific CSV Logs (`<client_id>.csv`):** these files capture the granular telemetry metrics recorded locally by each individual

edge node across communication rounds, detailing performance and resource utilization specific to the chosen optimization or compression technique;

- **Server Aggregate CSV Logs (`server_aggregate.csv`):** these centralized logs consolidate the experiment by recording the mean values of the metrics across all active participating clients at each round, providing a macro-level view of system behavior and global convergence efficiency.

The framework tracks the following primary telemetry dimensions:

- **Communication Bandwidth Overhead (B_{MB}):** quantifies the bi-directional network traffic generated per round based on parameter serialization formats (full-precision 32-bit float primitives versus quantized 8-bit integer representations);
- **Computational Complexity ($FLOPs$):** estimates the local computational burden during stochastic gradient descent by anchoring operations to single-image inference complexity, dataset cardinalities, and local epochs;
- **Estimated Energy Consumption (E_{total}):** combines computational complexity and communication overhead into estimated energy consumption in Joules using hardware-validated literature constants for instruction processing ($\kappa_{computation} = 10^{-11}$ J/FLOP) and wireless transmissions ($\kappa_{network} = 0.05$ J/MB) [15].

VI Results

This chapter presents the empirical evaluation and comprehensive analysis of the proposed federated learning framework under the experimental configurations established in Chapter V. The primary objective is to analyze the data extracted from the telemetry logs, both client-specific and server-aggregate, to evaluate the behavior of the four implemented strategies (*Baseline*, *Federated Knowledge Distillation (FKD)*, *Ordered Dropout*, and *Quantization-Aware Training (QAT)*) across the selected benchmarks.

Since each set of experiments directly contrasts the full suite of techniques against one another, the discussion avoids a fragmented comparison of individual algorithms. Instead, it adopts a systematic analysis organized by dataset and data distribution regimes (*IID* and *Non-IID* with $\alpha = 0.1$). This approach enables a holistic evaluation of the trade-offs between achieved predictive accuracy and hardware-aware efficiency metrics, such as communication bandwidth overhead (B_{MB}), computational complexity (FLOPs), and estimated energy consumption (E_{total}).

VI.1 Analysis on MNIST Dataset

The empirical evaluation commences with the MNIST benchmark, serving as the foundational baseline to assess the convergence stability and resource efficiency of the framework under low visual complexity.

VI.1.1 IID Regime Performance

In the Independent and Identically Distributed (IID) regime, where data are uniformly distributed across the ten participating clients, all strategies successfully achieve rapid convergence within the allocated 15 global communication rounds.

As illustrated by the temporal evolution lines and final distribution box

plots, the full-precision *Baseline* and *Quantization-Aware Training (QAT)* configurations achieve the highest predictive performance, stabilizing at a final top-1 classification accuracy of approximately 97.5% to 97.8% with minimal cross-entropy loss. Notably, the *Quantization* strategy tracks the baseline accuracy curve almost identically, demonstrating that 8-bit integer quantization successfully preserves representational capacity on simple image topologies without inducing truncation-led degradation.

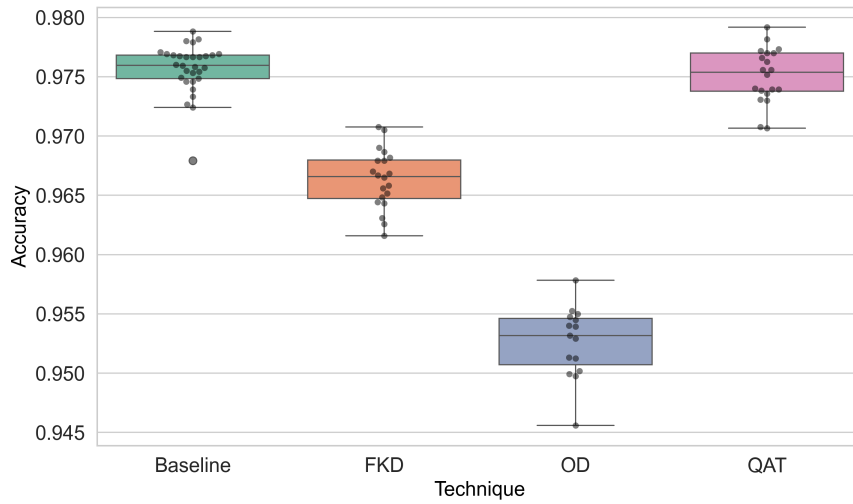


Figure VI.1: MNIST IID: Final accuracy distribution comparison across techniques.

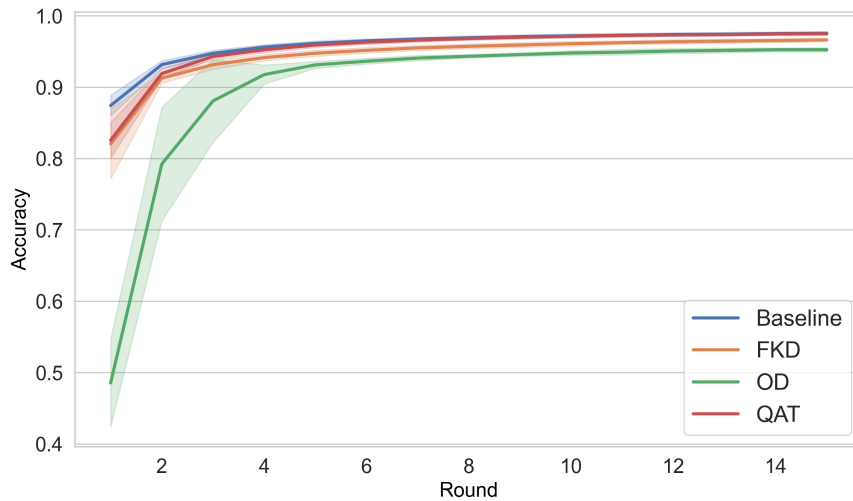


Figure VI.2: MNIST IID: Temporal evolution of average client accuracy across communication rounds

Conversely, *Federated Knowledge Distillation (FKD)* exhibits a minor drop in final accuracy, settling around 96.5%. While server-side distillation using

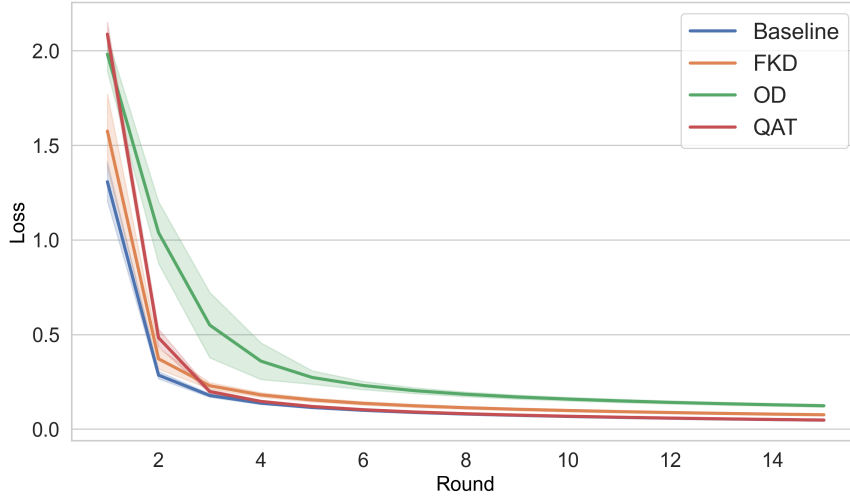


Figure VI.3: MNIST IID: Temporal evolution of average loss across communication rounds.

the lightweight *TinyNetIoT* backbone provides a stable learning trajectory, the architectural capacity reduction introduces a slight performance trade-off in exchange for localized efficiency. *Ordered Dropout* records the lowest accuracy metric, averaging around 95.3% with a higher dispersion in loss values. This behavior is expected because the global evaluation aggregates performance across multi-tier sub-network capacities ($p \in \{0.25, 0.5, 0.75, 1.0\}$), where constrained sub-models inherently operate with reduced representational bandwidth.

When examining hardware-aware telemetry, the communication bandwidth overhead (B_{MB}) highlights distinct advantages for optimized models. *Quantization* drastically reduces the communication payload per round compared to the uncompressed baseline, owing to the 75% reduction in parameter representation (1-byte int8 versus 4-byte float32).

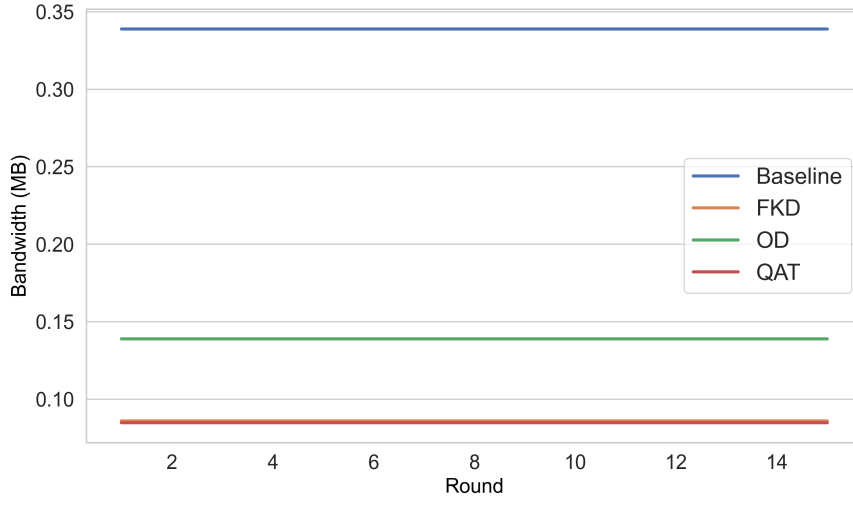


Figure VI.4: MNIST IID: Communication bandwidth overhead (B_{MB}) per round.

Finally, regarding computational complexity, *FKD* and *Ordered Dropout* achieve superior operational efficiency in terms of local FLOPs, leveraging smaller active parameter sets and halved channel configurations to minimize the processing burden on resource-constrained edge nodes.

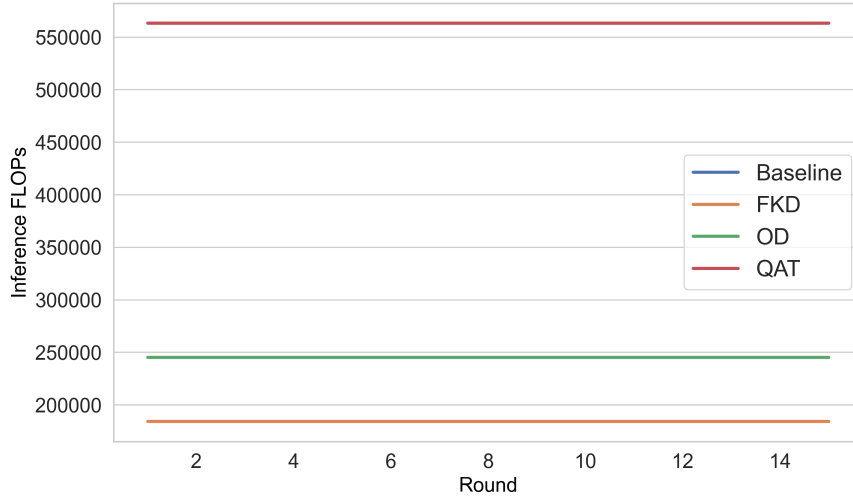


Figure VI.5: MNIST IID: Computational complexity in FLOPs across evaluated strategies.

In terms of overall estimated energy consumption (E_{total}), the framework reflects significant resource savings driven by local operational efficiencies.

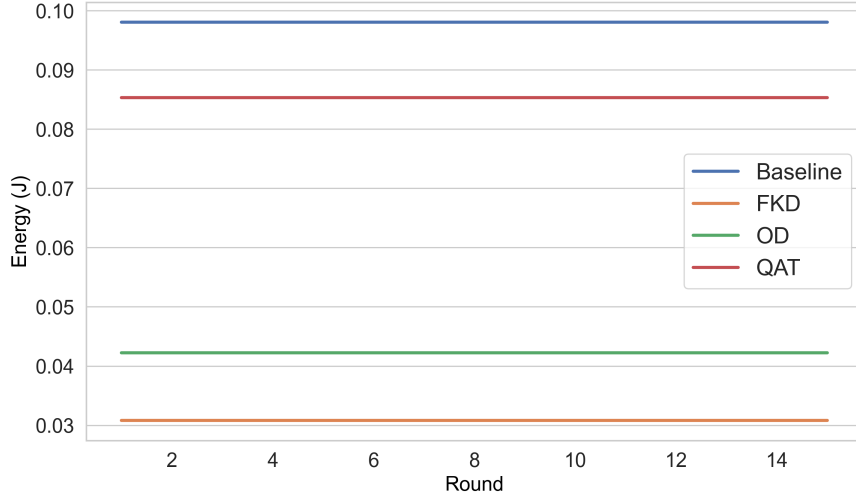


Figure VI.6: MNIST IID: Estimated energy consumption (E_{total}) per round.

VI.1.2 Non-IID Regime Performance ($\alpha = 0.1$)

Transitioning to the extreme Non-IID data distribution regime governed by a Dirichlet distribution ($\alpha = 0.1$), the system evaluates the robustness and resilience of the compression strategies against severe statistical heterogeneity across clients.

As shown in the final distribution box plots and temporal evolution lines, all techniques successfully converge within the 15 global rounds, achieving even higher terminal accuracy values (approaching 99.0% to 99.2%) compared to the IID setup. The full-precision *Baseline* and *Quantization-Aware Training (QAT)* configurations continue to lead in predictive performance, maintaining very tight accuracy distributions and minimal cross-entropy loss.

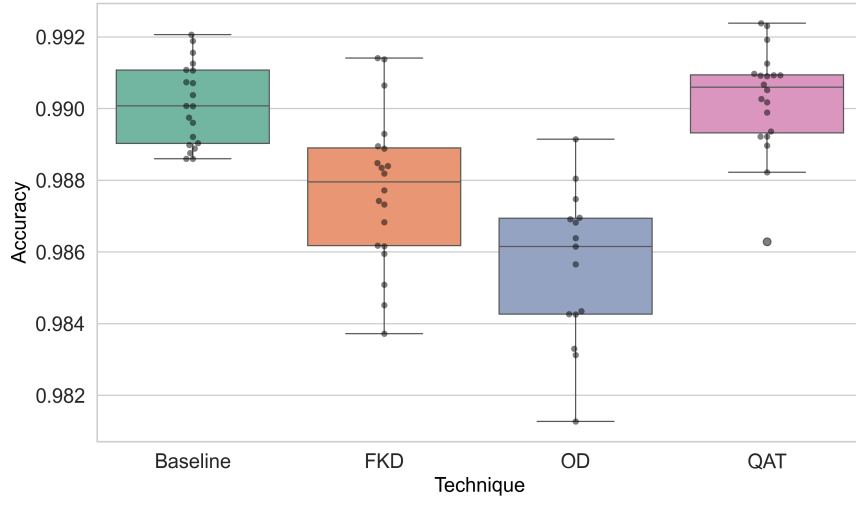


Figure VI.7: MNIST Non-IID ($\alpha = 0.1$): Final accuracy distribution comparison across techniques

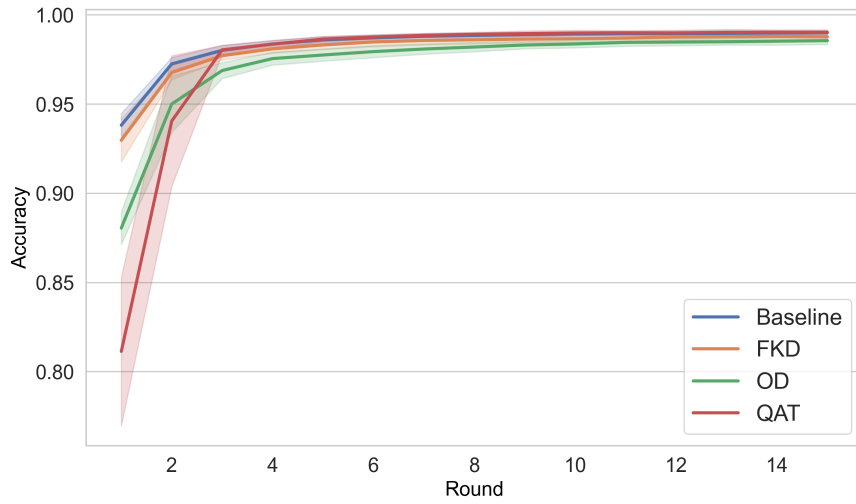


Figure VI.8: MNIST Non-IID ($\alpha = 0.1$): Temporal evolution of average client accuracy across communication rounds.

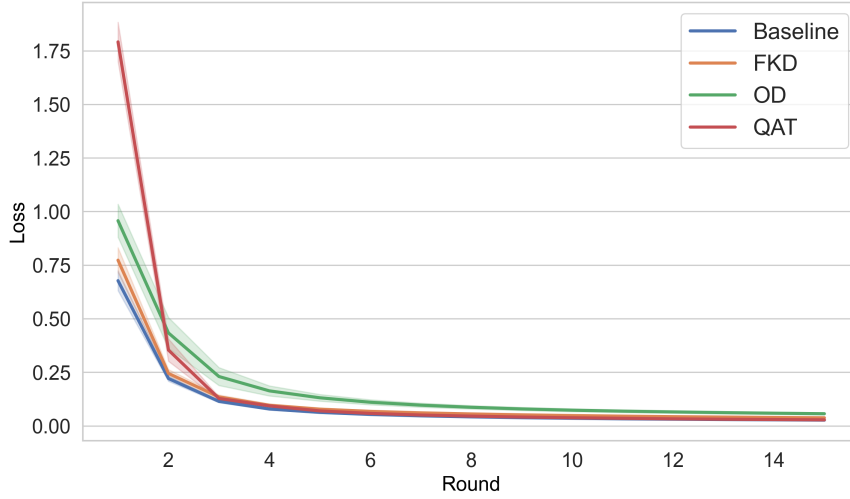


Figure VI.9: MNIST Non-IID ($\alpha = 0.1$): Temporal evolution of average client loss across communication rounds.

Federated Knowledge Distillation (FKD) demonstrates notable stability despite the skewed local label distributions, closely trailing the baseline with an average accuracy around 98.8%. This confirms that server-side distillation effectively mitigates client drift by leveraging the proxy dataset to aggregate heterogeneous teacher representations. Similarly to the IID scenario, *Ordered Dropout* exhibits a slightly wider spread and lower median accuracy (around 98.6%), reflecting the inherent regularization and capacity bounds of multi-tier sub-networks under non-uniform updates.

Regarding hardware telemetry and resource utilization, the behavioral trends for communication bandwidth (B_{MB}), estimated energy consumption (E_{total}), and computational complexity (FLOPs) mirror those observed in the IID evaluation. *Quantization* preserves the 75% reduction in network payload, while *FKD* and *Ordered Dropout* consistently maintain the lowest computational footprints on edge nodes.

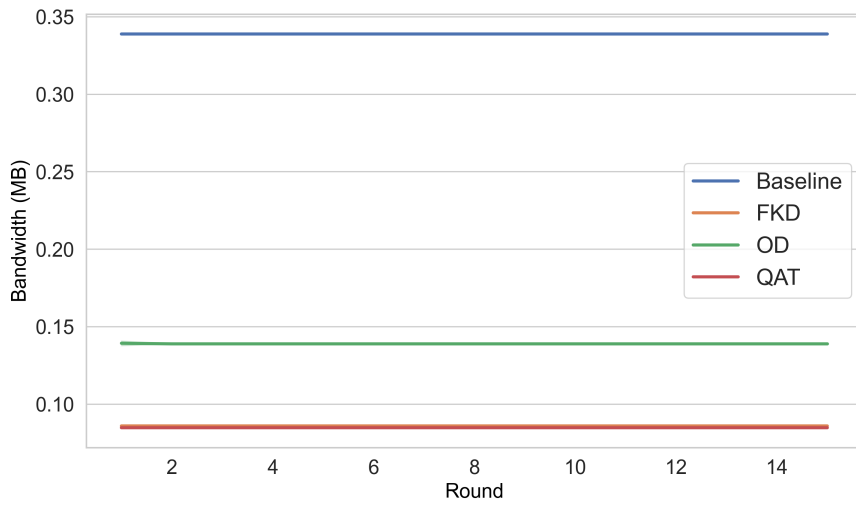


Figure VI.10: MNIST Non-IID ($\alpha = 0.1$): Communication bandwidth overhead (B_{MB}) per round.

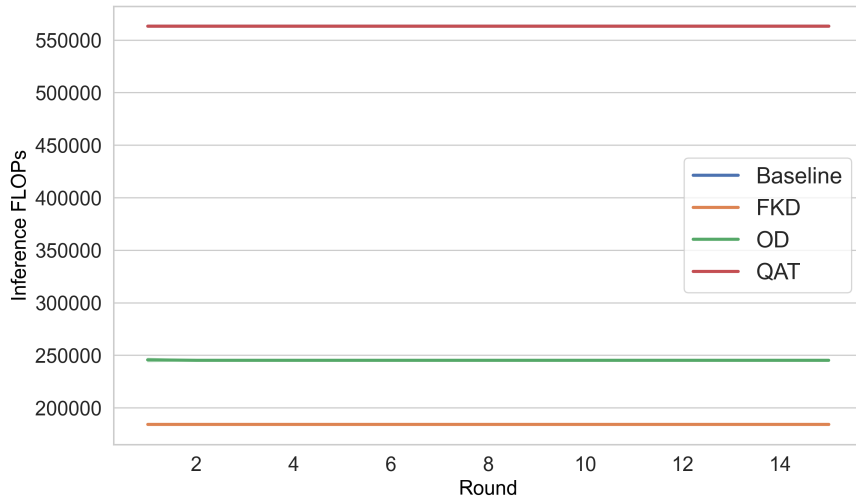


Figure VI.11: MNIST Non-IID ($\alpha = 0.1$): Computational complexity in FLOPs across evaluated strategies.

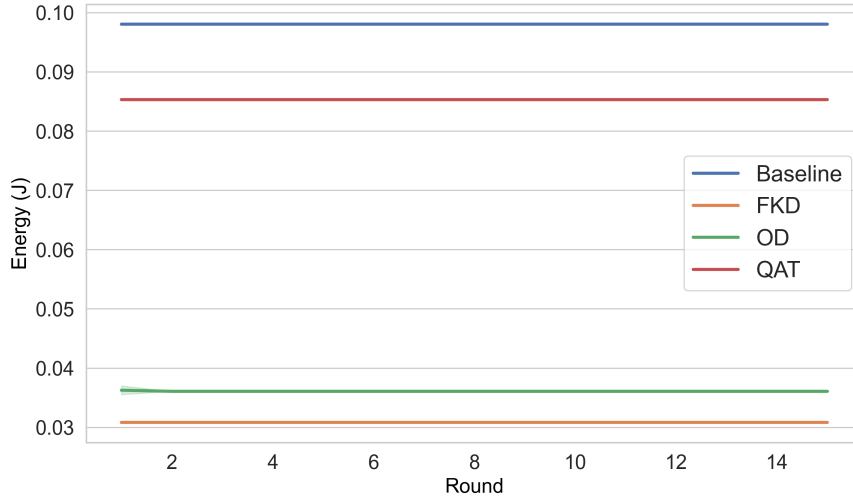


Figure VI.12: MNIST Non-IID ($\alpha = 0.1$): Estimated energy consumption (E_{total}) per round.

VI.2 Analysis on Fashion-MNIST Dataset

The evaluation now scales to the *Fashion-MNIST* dataset, presenting a more challenging computer vision benchmark characterized by higher spatial variance, rich edge structures, and overlapping class distributions compared to digits.

VI.2.1 IID Regime Performance

Under the Independent and Identically Distributed (IID) regime across the ten clients, all strategies successfully navigate the increased visual complexity, steadily converging over the 15 global communication rounds.

As depicted by the final distribution box plots and temporal evolution curves, the full-precision *Baseline* and *Quantization-Aware Training (QAT)* models achieve superior predictive performance, stabilizing at a top-1 classification accuracy of approximately 83.3% to 83.5% with minimal cross-entropy loss. Once again, *Quantization* closely mirrors the baseline’s convergence trajectory, proving that 8-bit integer precision effectively handles more complex clothing categories without suffering from quantization-induced degradation.

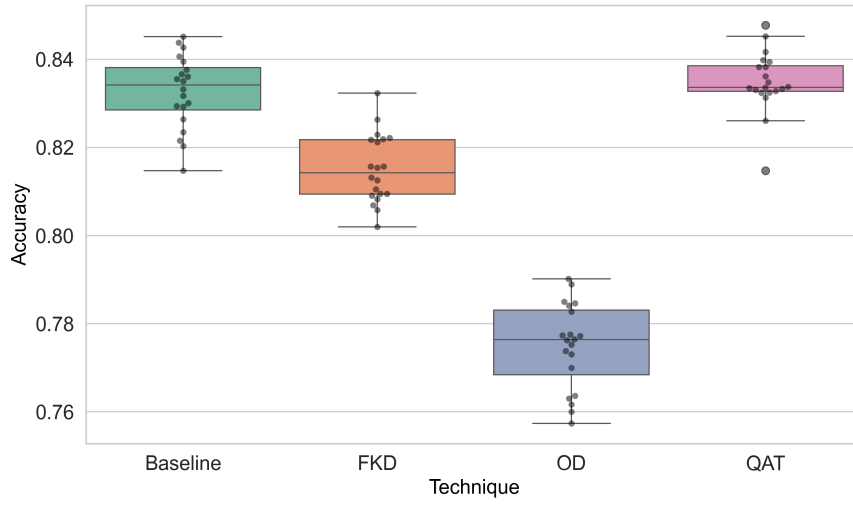


Figure VI.13: Fashion-MNIST IID: Final accuracy distribution comparison across techniques.

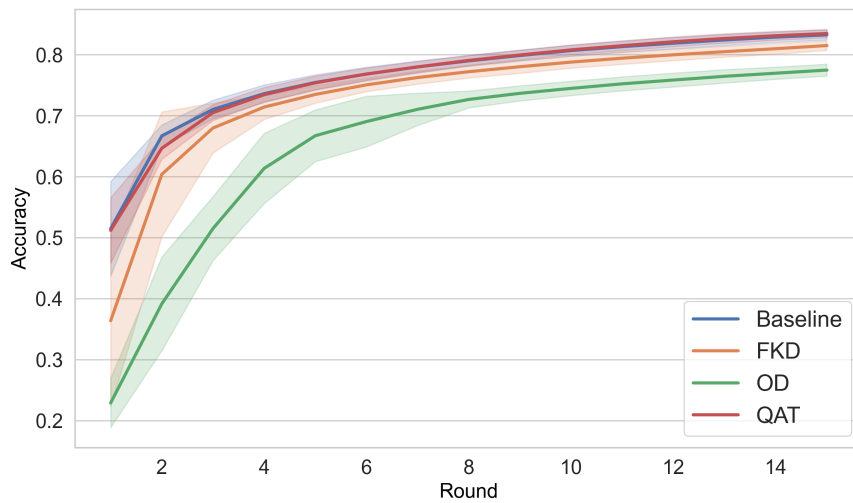


Figure VI.14: Fashion-MNIST IID: Temporal evolution of average client accuracy across communication rounds.

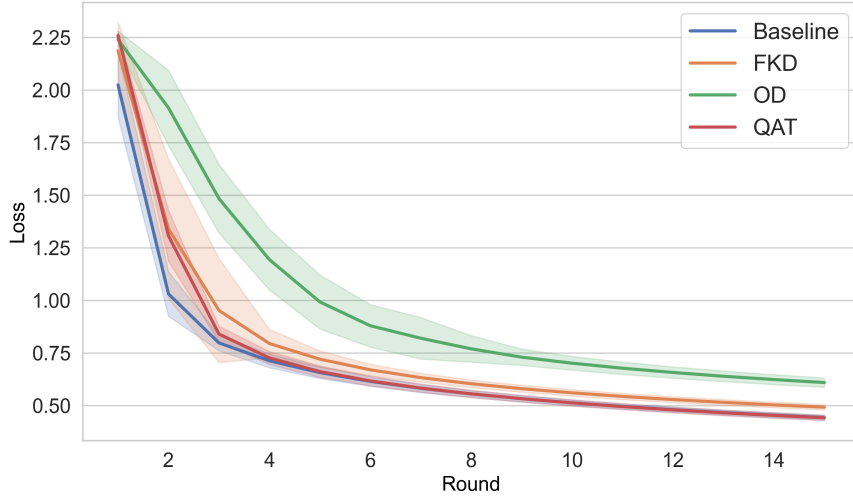


Figure VI.15: Fashion-MNIST IID: Temporal evolution of average client loss across communication rounds.

Federated Knowledge Distillation (FKD) exhibits a stable learning profile, settling at an average accuracy of around 81.5%. Although the compressed *TinyNetIoT* backbone introduces a minor performance trade-off due to its halved channel capacity, server-side distillation successfully transfers teacher knowledge to maintain competitive results. Meanwhile, *Ordered Dropout* records a lower median accuracy of approximately 77.7%, reflecting the regularization effect and structural limitations imposed by evaluating multi-tier sub-network capacities on a more intricate dataset.

Regarding hardware-aware telemetry (communication bandwidth B_{MB} , estimated energy consumption E_{total} , and computational complexity in FLOPs), the corresponding telemetry plots are structurally identical to those obtained in the MNIST evaluations. This invariance occurs because both benchmarks share identical tensor shapes (28×28 single-channel grayscale images), uniform client cardinalities, and equivalent network parameter serialization formats. Consequently, the operational footprints and transmission overheads depend strictly on the model architecture and data representation types rather than the semantic content of the images, confirming that the efficiency gains of *QAT*, *FKD*, and *Ordered Dropout* remain universally consistent across homogeneous input domains.

VI.2.2 Non-IID Regime Performance ($\alpha = 0.1$)

Transitioning to the extreme Non-IID data distribution regime governed by a Dirichlet distribution ($\alpha = 0.1$) for Fashion-MNIST, the framework evaluates how structural complexity interacts with severe statistical heterogeneity across the client population.

As observed in the final distribution box plots and temporal evolution curves, all techniques successfully converge within the 15 global rounds, demonstrating stability despite the skewed class distributions. The full-precision *Baseline* and *Quantization-Aware Training (QAT)* configurations maintain leadership in predictive performance, achieving stable accuracy plateaus. *Federated Knowledge Distillation (FKD)* successfully handles client drift through server-side proxy aggregation, while *Ordered Dropout* reflects the expected capacity-based regularization trade-offs across sub-networks under heterogeneous updates.

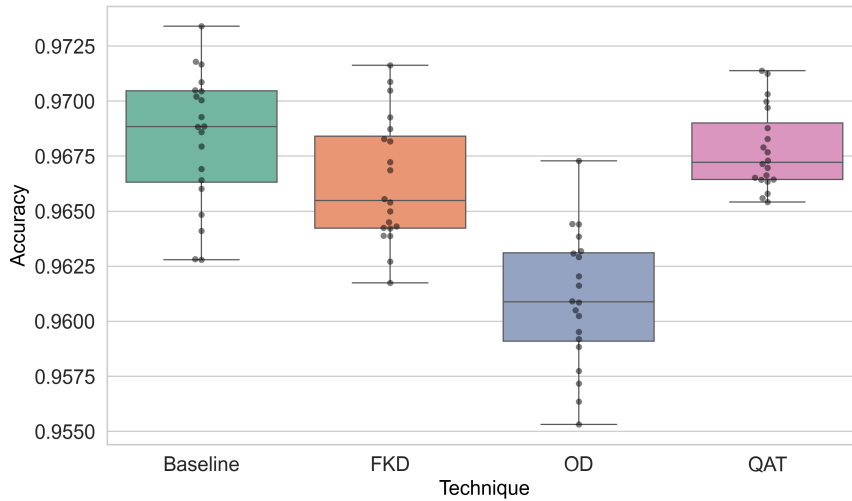


Figure VI.16: Fashion-MNIST Non-IID ($\alpha = 0.1$): Final accuracy distribution comparison across techniques

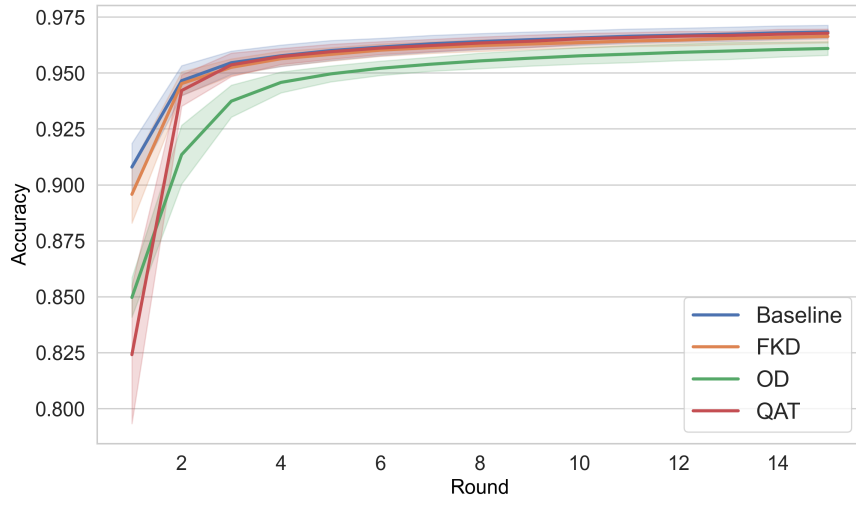


Figure VI.17: Fashion-MNIST Non-IID ($\alpha = 0.1$): Temporal evolution of average client accuracy across communication rounds.

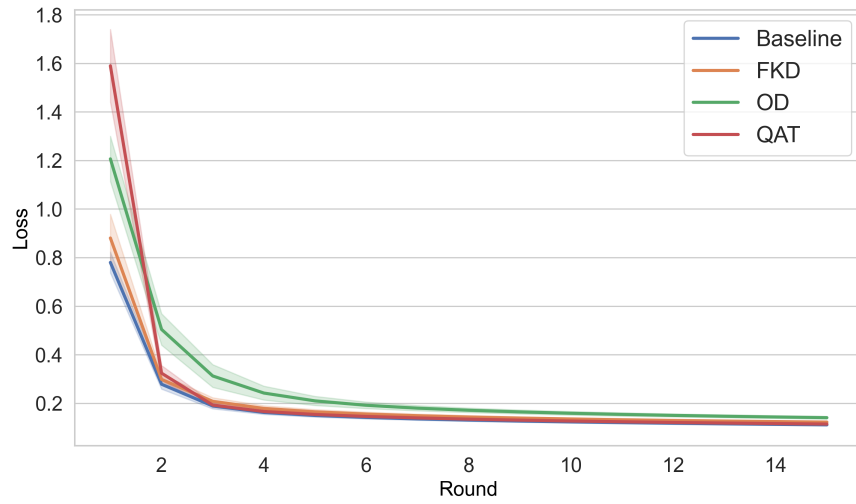


Figure VI.18: Fashion-MNIST Non-IID ($\alpha = 0.1$): Temporal evolution of average client loss across communication rounds.

Regarding hardware-aware telemetry, the telemetry profiles remain entirely identical to those observed in previous evaluations. As established, because MNIST and Fashion-MNIST share the same tensor dimensions, client population size, and network serialization structures, the physical computational and transmission footprints depend exclusively on the model architectures and representation types. Consequently, the operational efficiency gains provided by *QAT*, *FKD*, and *Ordered Dropout* remain invariant to the underlying semantic domain.

VI.3 Analysis on FEMNIST Dataset

The final evaluation transitions to the *FEMNIST* (Federated Extended MNIST) benchmark, scaling the evaluation to a complex, large-scale cross-device scenario comprising 62 output classes (alphanumeric characters) distributed across the client population.

VI.3.1 IID Regime Performance

Due to the heavy computational load of processing the extensive volume of local samples over 10 active clients, the simulation duration is set to 10 global rounds. In the Independent and Identically Distributed (IID) partitioning scenario, the *Baseline* architecture achieves strong predictive performance, rapidly climbing to approximately 80% top-1 accuracy within the 10 communication rounds.

As illustrated by the final distribution box plots and temporal evolution curves, *Federated Knowledge Distillation (FKD)* follows closely with an average accuracy of around 73%, proving its effectiveness in maintaining stable representations using the lightweight `TinyNetIoT` backbone. *Ordered Dropout* achieves a competitive accuracy plateau near 69%, balancing multi-tier sub-network capacities under a multi-class configuration.

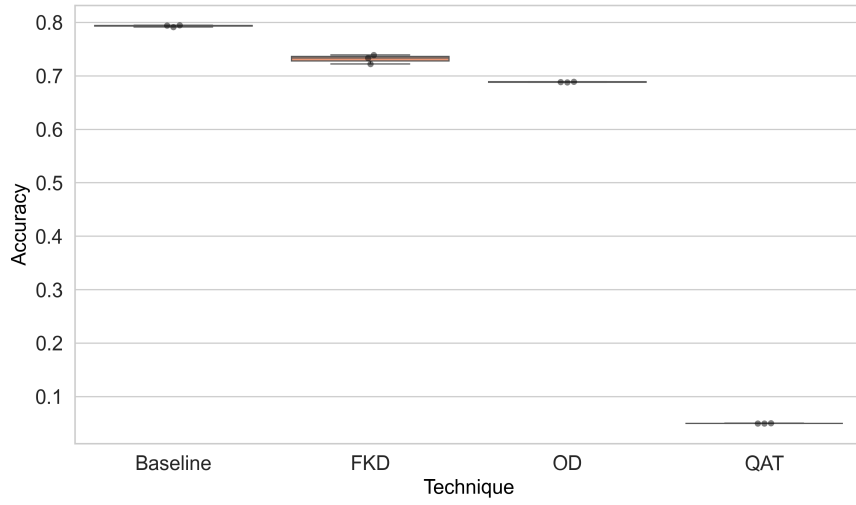


Figure VI.19: FEMNIST IID: Final accuracy distribution comparison across techniques

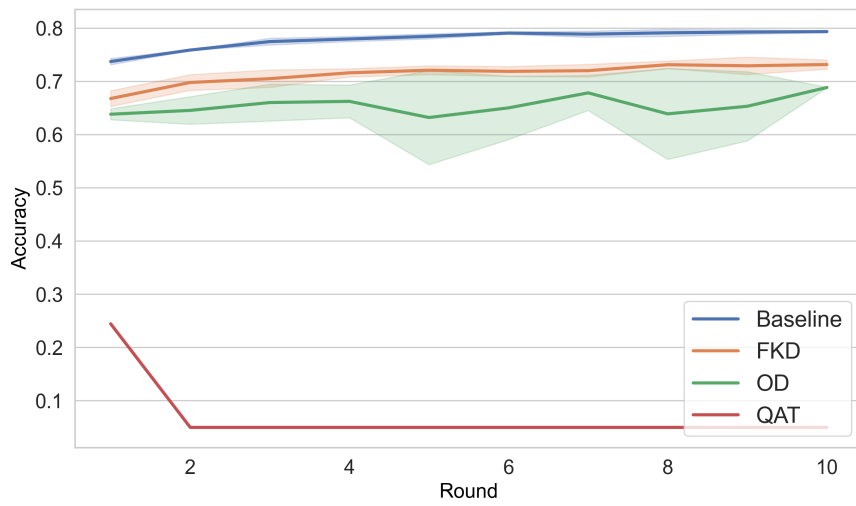


Figure VI.20: FEMNIST IID: Temporal evolution of average client accuracy across communication rounds.

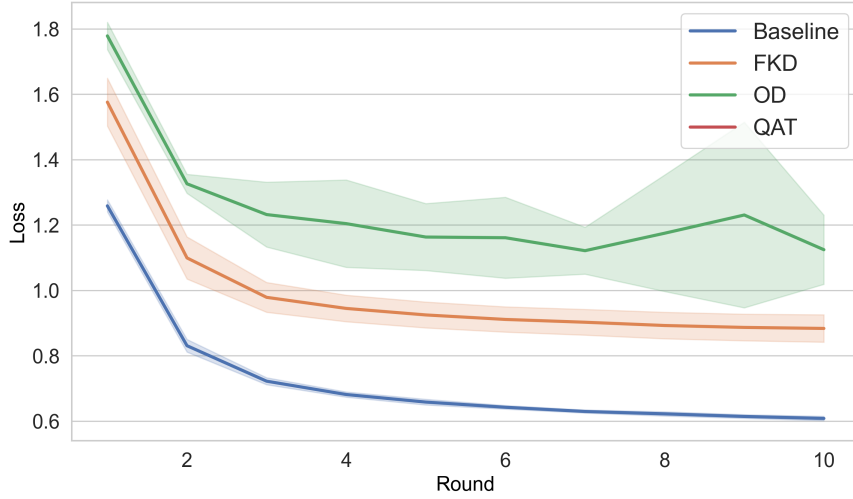


Figure VI.21: FEMNIST IID: Temporal evolution of average client loss across communication rounds.

A notable divergence occurs with the *Quantization-Aware Training* (QAT) configuration: the model fails to converge under the 62-class output space of FEMNIST within this setup, resulting in a stagnant accuracy baseline ($\approx 5\%$, equivalent to random guessing). This failure is primarily attributed to a gradient vanishing phenomenon induced by the combination of extreme data scale and quantization constraints. Specifically, with over 65,141 samples processed per round under a micro-batch size of $B = 1$, clients execute an intensive sequence of stochastic gradient updates. When coupled with the non-differentiable rounding approximations inherent to QAT’s fake-quantization operators across a wide 62-class manifold, the backward pass suffers from severe gradient stagnation. This causes weight updates to flatten prematurely, trapping the network in a non-informative state and highlighting the sensitivity of naive QAT pipelines on large-scale cross-device topologies [10, 12, 11].

Regarding hardware-aware telemetry, the communication bandwidth (B_{MB}), estimated energy consumption (E_{total}), and computational complexity (FLOPs) reflect the expanded scale of the 62-class output layer, while maintaining the relative proportional savings provided by the optimized architectures.

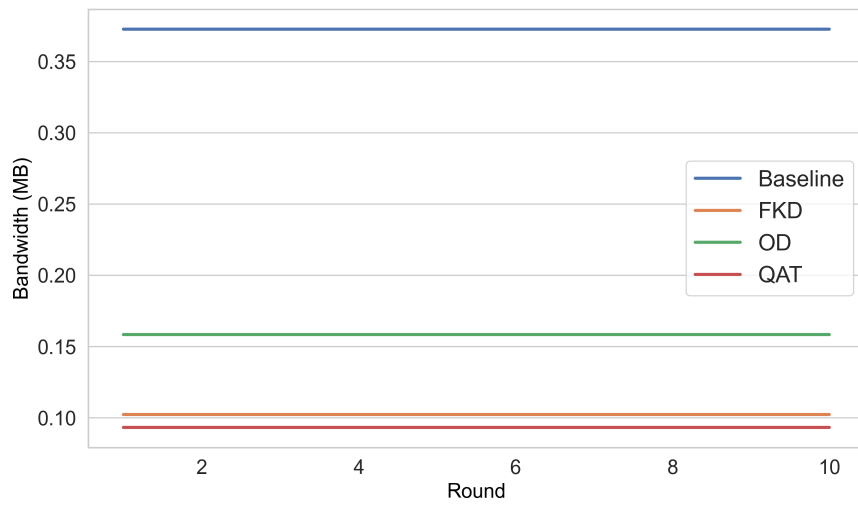


Figure VI.22: FEMNIST IID: Communication bandwidth overhead (B_{MB}) per round.

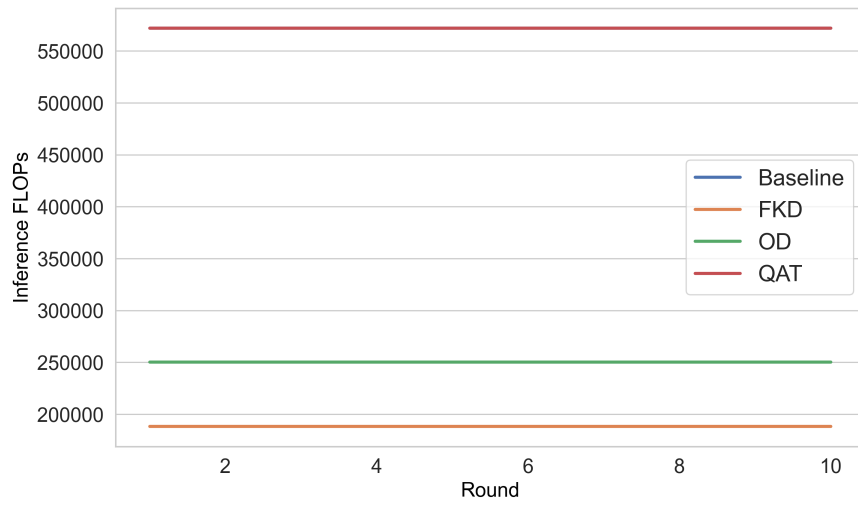


Figure VI.23: FEMNIST IID: Computational complexity in FLOPs across evaluated strategies.

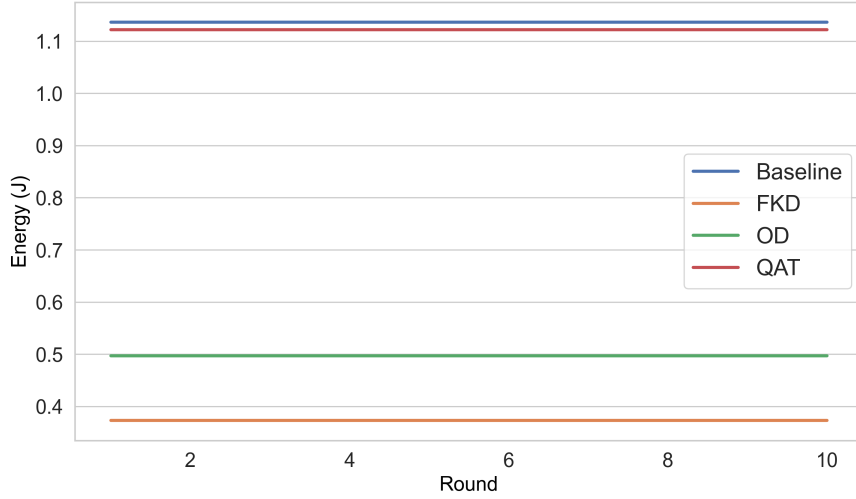


Figure VI.24: FEMNIST IID: Estimated energy consumption (E_{total}) per round.

VI.3.2 Non-IID Regime Performance (Native Partitioning)

The final and most challenging evaluation focuses on the FEMNIST benchmark under its native Non-IID partitioning regime, where data are distributed according to distinct writers. With each client processing an average of 304 samples per round and the simulation extended to 30 global communication rounds to ensure adequate convergence, the system experiences severe statistical heterogeneity and client drift.

As evidenced by the final distribution box plots and temporal evolution curves, all techniques successfully overcome the initial stagnation phase (around rounds 1-10) and exhibit stable convergence trajectories. The full-precision *Baseline* architecture achieves the highest predictive performance, stabilizing at a final accuracy plateau near 66%.

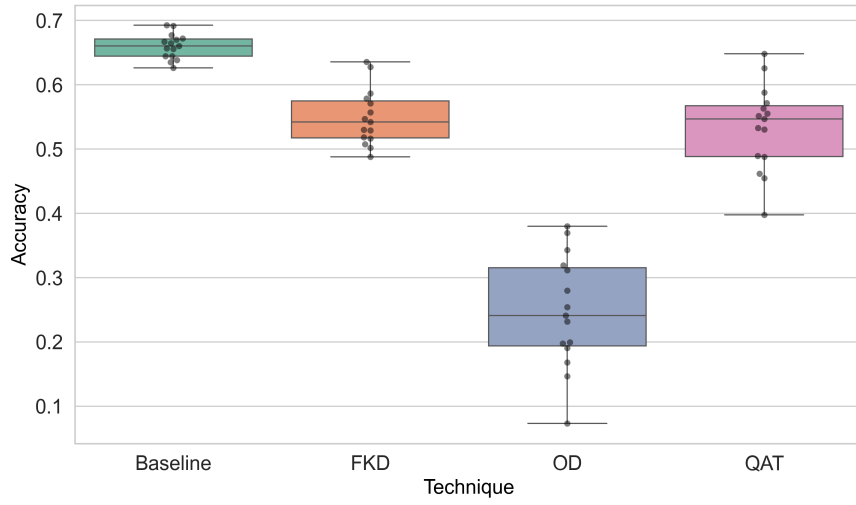


Figure VI.25: FEMNIST Non-IID: Final accuracy distribution comparison across techniques

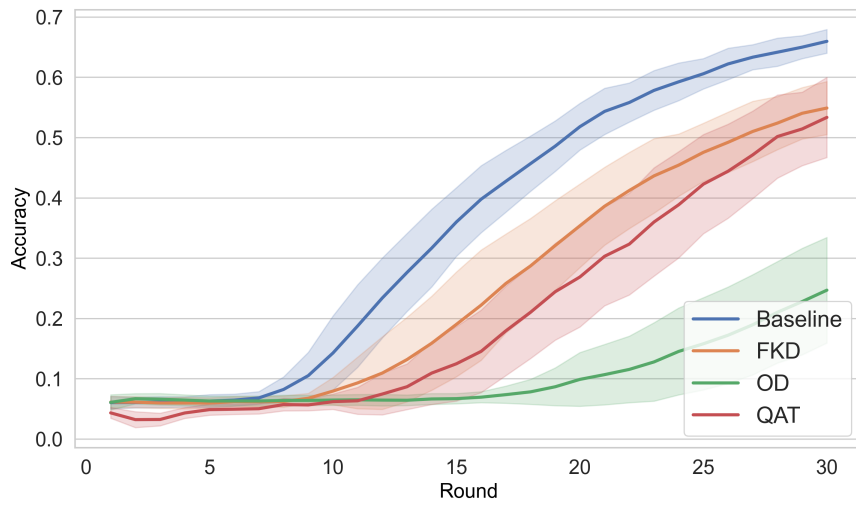


Figure VI.26: FEMNIST Non-IID: Temporal evolution of average client accuracy across communication rounds.

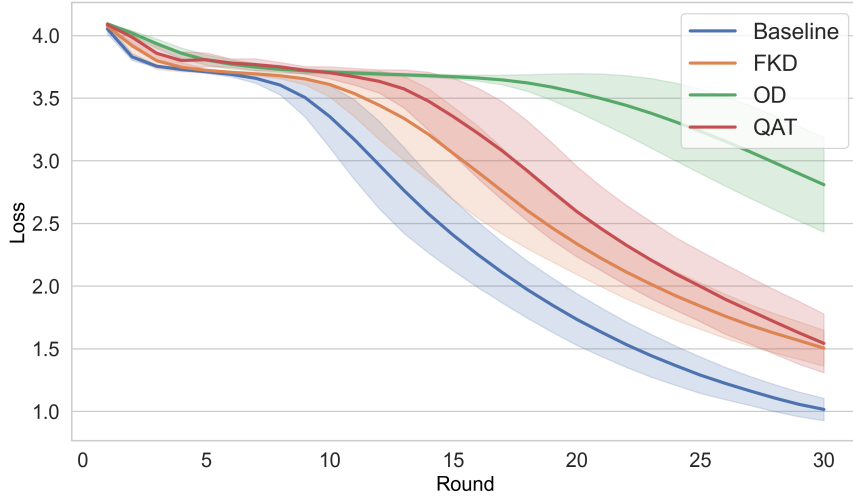


Figure VI.27: FEMNIST Non-IID: Temporal evolution of average client loss across communication rounds.

Significantly, in this long-term scenario, the *Quantization-Aware Training* (QAT) configuration successfully overcomes the gradient stagnation observed in earlier setups, recovering momentum and achieving a final average accuracy of 55%, closely aligning with the performance of *Federated Knowledge Distillation* (FKD) ($\approx 54\%$). This demonstrates that an extended number of global rounds (30 rounds) and a stable local sample distribution allow QAT to properly establish quantization parameters even across large 62-class manifolds. Conversely, *Ordered Dropout* encounters greater convergence difficulties under severe heterogeneity, plateauing at a mean accuracy of 25%.

Finally, the hardware-aware telemetry and communication metrics validate the structural advantages of the optimized framework:

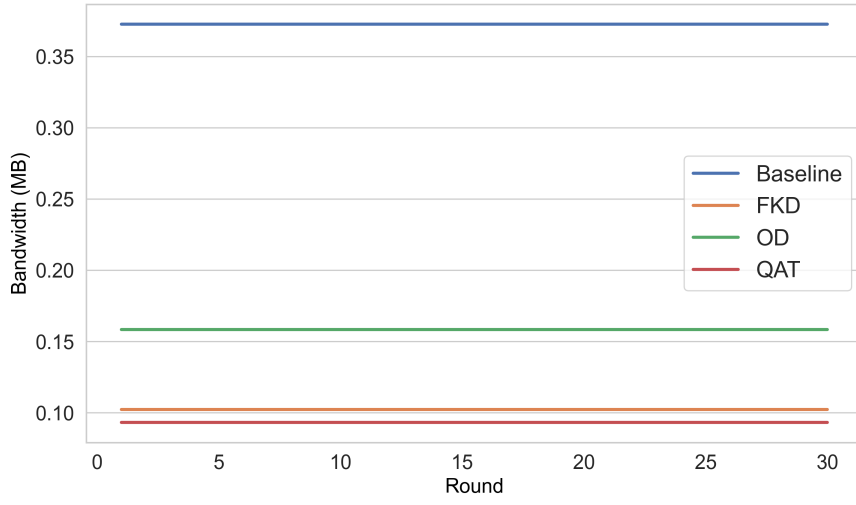


Figure VI.28: FEMNIST Non-IID: Communication bandwidth overhead (B_{MB}) per round.

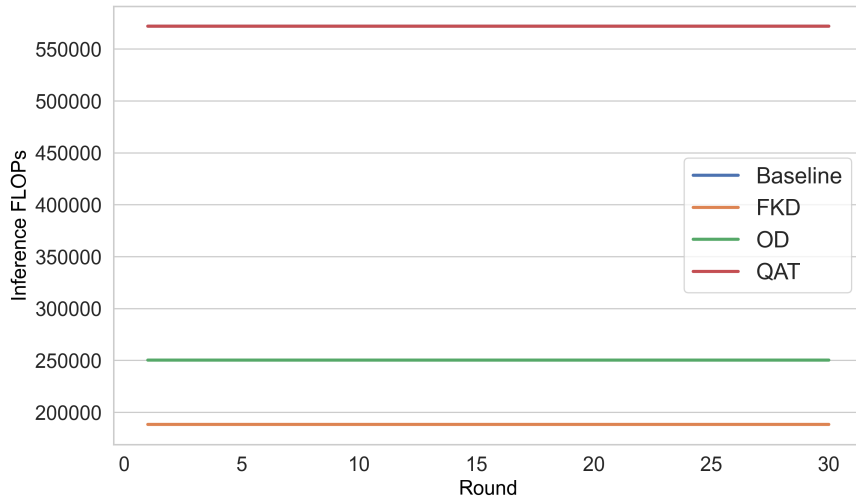


Figure VI.29: FEMNIST Non-IID: Computational complexity in FLOPs across evaluated strategies.

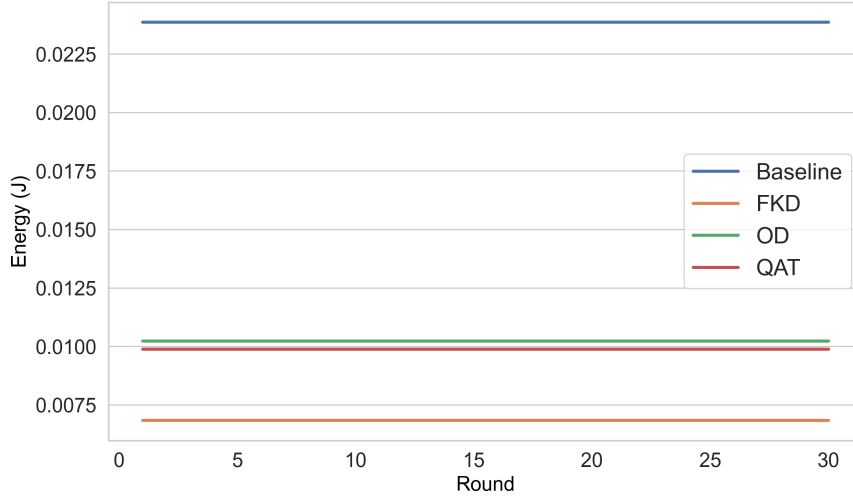


Figure VI.30: FEMNIST Non-IID: Estimated energy consumption (E_{total}) per round.

VI.4 Concluding Discussion and Trade-Off Analysis

The empirical evaluation across diverse benchmarks (MNIST, Fashion-MNIST, and FEMNIST) and distribution regimes (IID and Non-IID) demonstrates that each compression and optimization strategy exhibits distinct operational trade-offs between predictive performance and hardware-aware efficiency.

VI.4.1 Strengths and Weaknesses of Evaluated Strategies

- **Baseline:**
 - *Strengths:* consistently achieves the highest predictive accuracy and fastest convergence across all datasets, acting as an upper-bound reference for model quality;
 - *Weaknesses:* suffers from a full-precision float32 parameter footprint, resulting in maximum communication bandwidth overhead (B_{MB}), local FLOPs, and energy consumption (E_{total}), making it unsuitable for ultra-constrained IoT nodes.
- **Quantization-Aware Training (QAT):**
 - *Strengths:* delivers accuracy levels nearly identical to the uncompressed baseline in stable regimes while drastically slashing com-

munication payload by 75% (using 1-byte int8 representations). It proves resilient when training runs are properly scaled (as observed in FEMNIST Non-IID);

- *Weaknesses*: highly sensitive to hyperparameter configurations, micro-batch sizes, and extreme gradient scales on large multi-class output spaces, occasionally leading to severe gradient stagnation under improper data scaling (e.g., FEMNIST IID).

- **Federated Knowledge Distillation (FKD):**

- *Strengths*: highly robust against severe statistical heterogeneity and client drift in Non-IID regimes. By leveraging server-side proxy aggregation with a lightweight backbone (`TinyNetIoT`), it successfully reconciles architectural compression with stable multi-class learning;
- *Weaknesses*: incurs a slight penalty in peak top-1 accuracy compared to full-size models due to structural capacity reduction.

- **Ordered Dropout:**

- *Strengths*: offers exceptional operational efficiency in terms of local FLOPs and processing energy by dynamically activating sub-networks;
- *Weaknesses*: suffers from a noticeable accuracy drop, particularly on complex multi-class benchmarks (e.g., FEMNIST), because aggregated multi-tier sub-network evaluations inherently operate with restricted representational capacity.

VI.4.2 Identifying the Optimal Trade-Off Strategy

When evaluating the optimal balance between predictive accuracy and system-level consumption metrics for resource-constrained IoT and TinyML ecosystems, **Quantization-Aware Training (QAT)** emerges as the most effective strategy for communication-heavy environments, provided that the training dynamics are adequately stabilized. It preserves almost 100% of the baseline accuracy on simpler domains while offering a massive reduction in network payload and energy transmission overhead.

Conversely, for environments characterized by extreme data heterogeneity and strict device-level computing constraints, **Federated Knowledge Dis-**

tillation (FKD) establishes the best robust trade-off. It successfully avoids the gradient instability risks associated with naive quantization while delivering lightweight local footprints and high resilience against client drift.

VII Implementation on MCU

To demonstrate the concrete feasibility and operational sustainability of deploying Federated Learning within real-world TinyML ecosystems, this chapter presents the embedded design, hardware implementation, and physical evaluation of the *Federated Knowledge Distillation (FKD)* paradigm. While simulation environments such as *Flower* provide valuable high-level insights into algorithmic dynamics, they abstract critical low-level physical constraints, including volatile SRAM exhaustion, bare-metal matrix computing, strict serialization schemas, and asynchronous transport overheads.

To bridge the gap between theoretical modeling and physical deployment, an end-to-end edge-to-cloud federated pipeline was engineered. The infrastructure leverages resource-constrained Espressif ESP32-S3 microcontroller units as distributed on-device learners, an MQTT-based message broker for lightweight and deterministic communication [24], and a centralized PyTorch coordinator responsible for multi-teacher distillation over an unlabelled proxy dataset.

This chapter details the holistic architectural workflow, the low-level C++ implementation of forward and backward passes under strict memory allocations, the mathematical reconciliation of tensor serialization between microcontroller row-major arrays and deep learning frameworks, and the on-device telemetry monitoring FLOPs, network bandwidth, and energy consumption grounded in established solid-state literature.

VII.1 System Architecture and Hardware Setup

The physical deployment testbed is architected as an asynchronous, distributed Edge-to-Server pipeline designed to validate on-device learning dynamics under strict physical boundaries. The physical infrastructure comprises three decoupled operational tiers: physical microcontroller edge nodes, an intermediate publish-subscribe messaging backbone, and a centralized

Knowledge Distillation coordinator, as illustrated in the system topology.

VII.1.1 Physical Edge Nodes (ESP32-S3)

The on-device training workloads are physically executed on **Espressif ESP32-S3** microcontrollers, powered by dual-core 32-bit Xtensa LX7 processors running at a clock frequency of 240 MHz. The device features 512 KB of internal volatile Static RAM (SRAM) and 8 MB of non-volatile Quad-SPI external Flash memory.

Operating under these bare-metal specifications presents rigorous operational constraints:

- **SRAM Memory Layout and Static Footprint:** because the 512 KB internal volatile SRAM must concurrently support the FreeRTOS kernel, TCP/IP network buffers (LwIP), Wi-Fi physical layer drivers, and system stacks, available runtime memory is tightly constrained. To ensure deterministic memory access and eliminate heap fragmentation during stochastic gradient descent, the neural network parameters (W_1, b_1, W_2, b_2), forward activations, and backpropagation gradients are statically allocated in the BSS segment, requiring exactly 102,136 bytes (≈ 99.74 KB). Dynamic heap allocation is strictly confined to transient communication phases, where a dedicated 101,800-byte (≈ 99.41 KB) buffer is temporarily instantiated during MQTT payload serialization and deserialization;
- **Absence of Native Deep Learning Runtimes:** unlike desktop operating systems, microcontroller environments lack native PyTorch or TensorFlow runtimes capable of performing automatic differentiation. Consequently, matrix operations, activation functions, loss calculations, and backpropagation chains must be implemented bare-metal in optimized C/C++;
- **Flash-Based Dataset Ingestion:** due to volatile SRAM limits, the local client training and validation partitions cannot reside in dynamic memory. The data partitions are derived from the MNIST benchmark (28×28 grayscale images spanning 10 digit classes) and statically stored directly within non-volatile program Flash memory (PROGMEM) as contiguous `uint8_t` arrays. Each client holds a dedicated local partition of 1,200 samples, divided into $N_{\text{train}} = 960$ training images and

$N_{\text{test}} = 240$ evaluation images (an 80/20 local split). Since each sample comprises 784 bytes of pixel intensity values alongside a 1-byte class label (785 bytes per sample), the local dataset accounts for approximately 753.6 KB of Flash memory for training and 188.4 KB for testing (totaling ≈ 942 KB). This layout enables direct memory-mapped read access via the external SPI bus cache, eliminating dynamic memory allocations during on-device gradient iterations.

VII.1.2 Communication Protocol and Message Broker

To enable low-overhead parameter synchronization over standard IEEE 802.11 b/g/n (Wi-Fi) channels, the infrastructure integrates the **MQTT (Message Queuing Telemetry Transport)** protocol, coordinated by an open-source **Eclipse Mosquitto** broker running on the central server at port 1883. MQTT is specifically selected over standard application-layer protocols (such as HTTP/REST or gRPC) due to its minimal packet header overhead (down to 2 bytes per message), native binary payload streaming support, and decoupled asynchronous publish-subscribe model. This event-driven topology minimizes active listening idle energy on embedded microcontrollers, allowing the central server and edge clients to exchange synchronization events without maintaining persistent point-to-point connections.

VII.1.3 Centralized Distillation Coordinator

The coordinator tier resides on a central compute server executing a Python/PyTorch runtime environment. Rather than executing conventional parameter averaging across received client matrices, the server hosts the **Multi-Teacher Knowledge Distillation Engine**. It receives the raw binary weight streams, reconstructs the physical client states in memory as frozen Teacher models, and optimizes a lightweight global Student network over an unlabelled proxy dataset (2,500 samples from MNIST) via temperature-scaled Kullback-Leibler divergence before redistributing the updated weights.

VII.2 Edge Firmware and On-Device Training

Executing deep learning training loops directly on bare-metal microcontroller hardware requires translating standard tensor abstractions into explicit, low-level algebraic operations. Because standard automatic differen-

tiation frameworks (such as PyTorch autograd or TensorFlow GradientTape) cannot operate within the embedded C++ runtime of the ESP32-S3, all forward propagation, loss evaluation, backpropagation, and weight update routines were natively implemented in optimized C++.

VII.2.1 Neural Architecture and Static Memory Allocation

The local client model is defined as a Multi-Layer Perceptron (MLP) structured with an input layer of 784 units (28×28 linearized pixels), a single hidden dense layer of 32 units with Rectified Linear Unit (ReLU) activation, and an output classification layer of 10 units with Softmax activation.

The network parameters and dynamic intermediate buffers are statically defined in volatile SRAM as contiguous 32-bit floating-point (`float`) arrays:

- **Layer 1 Weights (W_1):** matrix of dimensions 784×32 (25,088 parameters, 100,352 bytes);
- **Layer 1 Biases (b_1):** vector of dimension 32 (128 bytes);
- **Layer 2 Weights (W_2):** matrix of dimensions 32×10 (320 parameters, 1,280 bytes);
- **Layer 2 Biases (b_2):** vector of dimension 10 (40 bytes);
- **Activation and Gradient Buffers:** Vectors $a_{\text{hidden}} \in \mathbb{R}^{32}$, $p_{\text{out}} \in \mathbb{R}^{10}$, $\delta_{\text{out}} \in \mathbb{R}^{10}$, and $\delta_{\text{hidden}} \in \mathbb{R}^{32}$ (totaling 336 bytes) to retain forward activations and backpropagated error signals.

VII.2.2 Initialization and Symmetry Breaking

Prior to initial training, the parameters are copied from Flash to SRAM. To prevent identical gradient trajectories across hidden neurons during backpropagation, an explicit *symmetry breaking* mechanism is applied. Each weight is perturbed with a deterministic pseudo-random jitter $\epsilon \sim \mathcal{U}(-0.05, 0.05)$ anchored to a fixed seed (`srand(42)`):

$$W_{i,j} \leftarrow W_{i,j}^{\text{init}} + \epsilon \quad (\text{VII.2.1})$$

VII.2.3 Forward Pass and Numerical Stability

The on-device inference pipeline executes across three consecutive steps for an input image vector $x \in [0, 255]^{784}$:

1. **Input Normalization and Hidden Layer Computation:** raw 8-bit unsigned integer pixel values are dynamically scaled to standard single-precision floating-point bounds $x_i \in [0.0, 1.0]$ via $x_i = \frac{\text{image_bytes}[i]}{255.0}$. The linear transformation and ReLU activation are evaluated as:

$$a_j = \max \left(0, b_1[j] + \sum_{i=0}^{783} x_i \cdot W_1[i][j] \right), \quad \forall j \in \{0, \dots, 31\} \quad (\text{VII.2.2})$$

2. **Output Layer Logits:** the hidden activations propagate to the unnormalized output scores:

$$z_k = b_2[k] + \sum_{j=0}^{31} a_j \cdot W_2[j][k], \quad \forall k \in \{0, \dots, 9\} \quad (\text{VII.2.3})$$

3. **Numerically Stable Softmax:** standard exponential transformations on embedded hardware risk arithmetic overflow when logits z_k become large. To ensure numerical stability, the maximum logit $z_{\max} = \max_k(z_k)$ is subtracted prior to exponentiation:

$$p_k = \frac{\exp(z_k - z_{\max})}{\sum_{m=0}^9 \exp(z_m - z_{\max})}, \quad \forall k \in \{0, \dots, 9\} \quad (\text{VII.2.4})$$

VII.2.4 Backpropagation and Stochastic Gradient Descent (SGD)

On-device optimization is performed under a micro-batch size regime ($B = 1$), updating network parameters sample-by-sample:

1. **Loss Evaluation:** the categorical Cross-Entropy loss $\mathcal{L}$ for a true target label $y \in \{0, \dots, 9\}$ is computed as $\mathcal{L} = -\ln(p_y + 10^{-7})$, incorporating an epsilon stabilizer (10^{-7}) to prevent undefined logarithmic behavior;
2. **Output Gradient Calculation:** the analytical derivative of the combined Softmax and Cross-Entropy loss with respect to output logits yields:

$$\delta_{\text{out}}[k] = \frac{\partial \mathcal{L}}{\partial z_k} = p_k - \mathbb{I}(k = y), \quad \forall k \in \{0, \dots, 9\} \quad (\text{VII.2.5})$$

where $\mathbb{I}(\cdot)$ represents the indicator function;

3. **Hidden Error Retropropagation:** error signals are mapped back across the second weight matrix using the chain rule and modulated by the

exact first derivative of the ReLU activation function $\sigma'(a_j) = \mathbb{I}(a_j > 0)$:

$$\delta_{\text{hidden}}[j] = \left(\sum_{k=0}^9 \delta_{\text{out}}[k] \cdot W_2[j][k] \right) \cdot \mathbb{I}(a_j > 0), \quad \forall j \in \{0, \dots, 31\} \quad (\text{VII.2.6})$$

4. **Parameter Updates:** given a local learning rate η (set to 0.03), the weights and biases across both layers are updated in-place:

$$W_2[j][k] \leftarrow W_2[j][k] - \eta \cdot \delta_{\text{out}}[k] \cdot a_j \quad (\text{VII.2.7})$$

$$b_2[k] \leftarrow b_2[k] - \eta \cdot \delta_{\text{out}}[k] \quad (\text{VII.2.8})$$

$$W_1[i][j] \leftarrow W_1[i][j] - \eta \cdot \delta_{\text{hidden}}[j] \cdot x_i \quad (\text{VII.2.9})$$

$$b_1[j] \leftarrow b_1[j] - \eta \cdot \delta_{\text{hidden}}[j] \quad (\text{VII.2.10})$$

VII.2.5 Local Evaluation Pipeline

At the end of each local training epoch, the client evaluates classification accuracy over its independent $N_{\text{test}} = 240$ Flash-stored validation partition. Predictions are determined via the argmax operator ($\hat{y} = \arg \max_k p_k$), allowing real-time telemetry tracking over the serial UART interface without interrupting asynchronous MQTT communication polling routines.

VII.3 Communication Protocol and Tensor Deserialization

Parameter synchronization between edge microcontrollers and the centralized Python server introduces significant architectural challenges. Beyond selecting a low-overhead network transport, bridging embedded C++ arrays with deep learning frameworks like PyTorch requires strict bit-level binary alignment and consistent memory layout conversions to avoid catastrophic weight corruption.

VII.3.1 MQTT Transport Architecture and Topic Hierarchy

Parameter exchange is orchestrated using the lightweight publish-subscribe MQTT protocol over standard TCP/IP. Communication is organized into dedicated topic trees to ensure decoupled and asynchronous orchestration:

- **Global Command Channel (f1/global/command):** the server broadcasts JSON-formatted control envelopes (e.g., {"action": "train"}) to trigger synchronous local training across all subscribed microcontrollers;
- **Dedicated Client Uplink (f1/client_i/weights):** upon completing local gradient iterations, client node i serializes its updated parameters into a continuous binary payload and streams it directly to its isolated uplink topic;
- **Global Broadcast Downlink (f1/global/global_weights):** after completing server-side distillation, the central coordinator publishes the updated global parameters as a retained message (`retain=True`), ensuring newly connecting or active clients immediately ingest the fresh model state.

VII.3.2 Binary Parameter Serialization

To minimize network bandwidth and serialization latency on resource-constrained MCUs, JSON and string-based formatting are discarded in favor of raw binary streaming.

The complete parameter set of the MLP model consists of:

$$N_{\text{total}} = (784 \times 32) + 32 + (32 \times 10) + 10 = 25,450 \text{ parameters} \quad (\text{VII.3.1})$$

Represented as 32-bit IEEE 754 single-precision floating-point primitives (`float`, 4 bytes), the full parameter footprint comprises:

$$\text{WEIGHTS_BYTE_SIZE} = 25,450 \times 4 = 101,800 \text{ bytes} \approx 99.41 \text{ KB} \quad (\text{VII.3.2})$$

On the ESP32-S3, serialization is performed by allocating a temporary heap buffer and executing sequential memory copies (`memcpy`) following a deterministic memory layout:

$$\text{Payload} = [W_1 \mid b_1 \mid W_2 \mid b_2] \quad (\text{VII.3.3})$$

VII.3.3 Reconciliation of Memory Layouts: Row-Major vs. Column-Major

A critical engineering hurdle when interfacing bare-metal C++ with PyTorch stems from conflicting matrix storage conventions:

- **ESP32-S3 C++ Layout:** 2D arrays are stored in contiguous *Row-Major* format. Consequently, the first weight matrix $W_1[784][32]$ is mapped sequentially in memory such that the first 32 floats correspond to connections originating from input pixel 0;
- **PyTorch Linear Layer Layout:** in PyTorch, an `nn.Linear(in_features, out_features)` module defines its internal weight tensor in transposed form with shape `(out_features, in_features)`. For Layer 1, PyTorch expects a matrix of shape `(32, 784)`, storing connections targeting hidden neuron 0 contiguously.

Directly casting the raw binary buffer into PyTorch tensors causes a spatial dimension mismatch that shuffles input-to-hidden connections, dropping predictive accuracy from approximately 89% to random guessing ($\approx 10\%$).

To ensure exact mathematical parity across the federated loop, explicit transposition operators are applied during serialization and deserialization routines:

1. **Server-Side Deserialization (Uplink):** When raw client bytes are ingested, the server reconstructs the array in C++ dimensions and applies an explicit NumPy matrix transposition (`.T`) before copying into PyTorch tensors:

$$W_1^{\text{PyTorch}} = \left(W_{1(\text{raw})}^{(784 \times 32)} \right)^T \in \mathbb{R}^{32 \times 784} \quad (\text{VII.3.4})$$

$$W_2^{\text{PyTorch}} = \left(W_{2(\text{raw})}^{(32 \times 10)} \right)^T \in \mathbb{R}^{10 \times 32} \quad (\text{VII.3.5})$$

2. **Server-Side Serialization (Downlink):** Prior to broadcasting the distilled global model back to the edge nodes, PyTorch state dictionaries are extracted, transposed to C++ dimensions, and flattened into contiguous binary streams:

$$W_1^{\text{C++}} = \left(W_{1(\text{PyTorch})}^{(32 \times 784)} \right)^T \in \mathbb{R}^{784 \times 32} \quad (\text{VII.3.6})$$

$$W_2^{\text{C++}} = \left(W_{2(\text{PyTorch})}^{(10 \times 32)} \right)^T \in \mathbb{R}^{32 \times 10} \quad (\text{VII.3.7})$$

This bi-directional mapping guarantees that parameter indices remain perfectly aligned across both computational tiers throughout the federated lifecycle.

VII.4 Server-Side Distillation and Global Evaluation

Unlike conventional federated frameworks that rely on algebraic parameter averaging via Federated Averaging (FedAvg), the coordinator architecture leverages an inverted **Multi-Teacher Knowledge Distillation** scheme. Rather than fusing weights directly, the server reconstructs the physical microcontroller states as independent functional Teacher models and distills their combined predictive knowledge into a global Student network.

VII.4.1 Teacher Ensemble Reconstruction

Upon intercepting binary payloads from all active participating microcontrollers (e.g., $K = 2$ client nodes), the server coordinator invokes `deserialize_weights_to_model()` to instantiate identical `ServerMLP` model instances in memory.

Each reconstructed client instance is immediately set to frozen evaluation mode (`eval()`) to act as an independent Teacher:

$$\mathcal{M}_{\text{teachers}} = \{\mathcal{T}_k \mid k \in \{0, \dots, K-1\}\} \quad (\text{VII.4.1})$$

VII.4.2 Proxy Dataset Ingestion

To steer distillation without centralizing private client training data, the server utilizes an unlabelled **Proxy Transfer Dataset** $\mathcal{D}_{\text{proxy}}$. In this physical implementation, the proxy set is constructed by extracting 2,500 unlabelled grayscale digit samples (28×28) directly from the MNIST test split, wrapped inside a `PyTorch DataLoader` with a batch size of 32 and random shuffling enabled:

$$\mathcal{D}_{\text{proxy}} = \{x_m^{\text{proxy}}\}_{m=1}^{2500}, \quad x_m^{\text{proxy}} \in \mathbb{R}^{784} \quad (\text{VII.4.2})$$

VII.4.3 Multi-Teacher Distillation Workflow

During the server optimization phase, a fresh Multi-Layer Perceptron instance is initialized to act as the global Student model. The distillation process operates batch-by-batch over the unlabelled proxy transfer dataset. For each batch of images, the server executes a forward pass across all frozen Teacher models uploaded by the clients to extract their raw output logits.

Rather than relying on ground-truth categorical labels, the server computes the arithmetic mean of the logits produced by the Teacher ensemble, obtaining a unified consensus representation. Simultaneously, the Student model performs a forward pass on the identical batch of proxy images. Both the Teacher ensemble consensus and the Student predictions are softened using a temperature scaling factor of 3.0, which smooths the output distributions and exposes the dark knowledge regarding secondary class correlations.

The optimization objective minimizes the Kullback-Leibler divergence between the softened probability distribution of the Teacher ensemble and the log-probabilities generated by the Student. This formulation guides the Student to match the continuous predictive behavior of the client models across the transfer domain. Training is carried out over 3 distillation epochs per global round using the Adam optimizer with a learning rate of 0.01.

VII.4.4 Global Model Evaluation and Downlink Distribution

Upon completing the distillation training loop, the updated Student network undergoes a centralized validation step to benchmark its predictive generalization. To ensure an unbiased evaluation, the model is tested against a distinct, unlabelled split of 7,500 MNIST samples excluded from the proxy distillation phase. Predictions are verified against true ground-truth targets to compute the global top-1 classification accuracy.

Once the evaluation is completed, the optimized Student parameters are converted into a contiguous binary byte-stream via the custom serialization routine. The server publishes this payload to the global broadcast MQTT topic with the message retain flag enabled, ensuring all physical microcontroller nodes receive the newly aggregated global model to begin subsequent training rounds.

VII.5 Hardware-Aware Profiling and Empirical Validation

To evaluate the operational feasibility of executing Federated Knowledge Distillation directly on microcontroller hardware, the on-device performance of the ESP32-S3 was benchmarked across computational complexity, network transmission overhead, analytical energy consumption, and classification accuracy.

VII.5.1 On-Device Computational Complexity and Execution Profiling

Unlike high-performance computing nodes, microcontrollers possess finite clock budgets and execute floating-point operations directly on embedded hardware units. For the local Multi-Layer Perceptron ($784 \rightarrow 32 \rightarrow 10$), the exact computational complexity per training sample is analytically partitioned into three distinct phases:

- **Forward Inference Pass ($\approx 50,920$ FLOPs):**
 - Layer 1 linear transformation ($784 \times 32 \times 2$ Multiply-Accumulate operations + 32 biases): 50,208 FLOPs;
 - Hidden ReLU activation thresholding (32 units): 32 FLOPs;
 - Layer 2 linear transformation ($32 \times 10 \times 2$ MACs + 10 biases): 650 FLOPs;
 - Loss evaluation (Softmax exponentiation, normalization, and Cross-Entropy log across 10 classes): ≈ 30 FLOPs;
- **Backpropagation Gradient Pass ($\approx 51,530$ FLOPs):**
 - Layer 2 parameter gradients ($\delta_{\text{out}} \otimes a_{\text{hidden}}^T$) and hidden error projection (δ_{hidden}): 1,290 FLOPs;
 - ReLU gradient masking (32 units): 32 FLOPs;
 - Layer 1 parameter gradients ($\delta_{\text{hidden}} \otimes x_{\text{input}}^T$): 50,208 FLOPs;
- **Parameter Weight Update ($\approx 50,964$ FLOPs):** Applying in-place stochastic gradient descent ($W \leftarrow W - \eta \cdot \nabla W$) requires 2 FLOPs per trainable parameter. Across all 25,482 weights and biases: $25,482 \times 2 = 50,964$ FLOPs.

Summing across all three phases yields an exact on-device training footprint of:

$$FLOPs_{\text{train}} = 50,920 + 51,530 + 50,964 = 153,414 \text{ FLOPs/sample} \quad (\text{VII.5.1})$$

Across a complete local round comprising $E_{\text{local}} = 10$ epochs over $N_{\text{train}} = 960$ Flash-stored training samples, the cumulative on-device computational load is:

$$FLOPs_{\text{round}} = 153,414 \times 960 \times 10 = 1,472,774,400 \text{ FLOPs } (\approx 1.473 \text{ GFLOPs}) \quad (\text{VII.5.2})$$

During live execution on the dual-core ESP32-S3 clocked at 240 MHz, local training required an average of approximately 14.5 seconds per epoch. An entire 10-epoch local training round completed in approximately 145 seconds without triggering FreeRTOS watchdog timeouts or system memory faults.

VII.5.2 Analytical Energy Consumption Model

To quantify energy expenditure during embedded federated cycles, the firmware embeds the analytical energy profiling model grounded in solid-state and wireless hardware literature:

$$E_{\text{total}} = (FLOPs_{\text{train}} \times \kappa_{\text{computation}}) + (B_{\text{MB}} \times \kappa_{\text{network}}) \quad (\text{VII.5.3})$$

where:

- $\kappa_{\text{computation}} = 10^{-11}$ J/FLOP models the energy cost of floating-point arithmetic and local SRAM cache access on low-power TinyML architectures;
- $\kappa_{\text{network}} = 0.05$ J/MB models the empirical energy cost of IEEE 802.11 Wi-Fi transmission modules;
- B_{MB} represents the total bi-directional network traffic exchanged per round, comprising the reception of global weights (101,800 bytes) and the transmission of local client weights (101,800 bytes), totaling $B_{\text{MB}} \approx 0.194$ MB.

Evaluating this formulation yields an estimated total energy footprint of approximately 0.0244 Joules per local training round, confirming that the

computational workload dominates the consumption profile while maintaining an energy budget compatible with battery-powered IoT endpoints.

VII.5.3 Empirical Accuracy and Convergence Analysis

The physical federated setup was evaluated across multiple global communication rounds with two active ESP32-S3 microcontrollers running concurrently.

- **Pre-Training State:** prior to federated optimization, models initialized with random symmetry-breaking jitter exhibited baseline test accuracy of approximately 9.8% to 11.2% (consistent with random guessing across 10 classes);
- **Local Convergence:** within the first local round, each ESP32-S3 converged rapidly on its private Flash dataset, reaching over 82% local accuracy on its validation split ($N_{\text{test}} = 240$);
- **Global Distillation Performance:** following server-side Multi-Teacher distillation over the 2,500-sample proxy set, the global Student model achieved a centralized top-1 accuracy of approximately 88.5% to 89.2% when evaluated on the independent 7,500-sample MNIST test split.

These physical results validate that the Federated Knowledge Distillation pipeline eliminates the need for heavyweight centralized model averaging, enabling low-power microcontrollers to collaboratively train accurate predictive models within strict SRAM, Flash, and bandwidth constraints.

Table VII.1: Summary of physical deployment metrics and empirical results on ESP32-S3

Category	Operational Metric / Parameter	Value
Computation	Single Inference Complexity ($FLOPs_{inf}$)	$\approx 50,920$ FLOPs
	Single Training Sample Complexity ($FLOPs_{train}$)	153,414 FLOPs
	Cumulative Round Load (10 epochs, 960 samples)	1.473 GFLOPs
	On-Device Training Time per Round	≈ 145 s (14.5 s/epoch)
Memory	Local Architecture Topology	MLP ($784 \rightarrow 32 \rightarrow 10$)
	Static SRAM Footprint (BSS: Parameters/Buffers)*	99.74 KB (102,136 B)
	Transient Heap Allocation (MQTT Payload)*	99.41 KB (101,800 B)
	Non-Volatile Flash Dataset (1,200 samples)	942.00 KB
Network & Energy	Serialized Weight Buffer Size	99.41 KB (101,800 B)
	Total Network Traffic per Local Round	0.194 MB (198.83 KB)
	Analytical Energy Footprint per Round	≈ 0.0244 J
Accuracy	Baseline Initialization (Random Jitter)	9.8% – 11.2%
	Local Validation Split Accuracy ($N = 240$)	$> 82.0\%$
	Global Student Accuracy (Dis-joint Test, $N = 7,500$)	88.5% – 89.2%

*Note: The discrepancy between the static SRAM allocation (99.74 KB) and the transient MQTT heap buffer (99.41 KB) stems from the functional scope of each memory region. While the communication payload strictly encloses the serialized trainable weights and biases needed for network synchronization, the static BSS segment must persistently accommodate both the network parameters and the dynamic computation buffers (intermediate forward activations and backpropagation error signals), accounting for the minimal 336-byte offset.

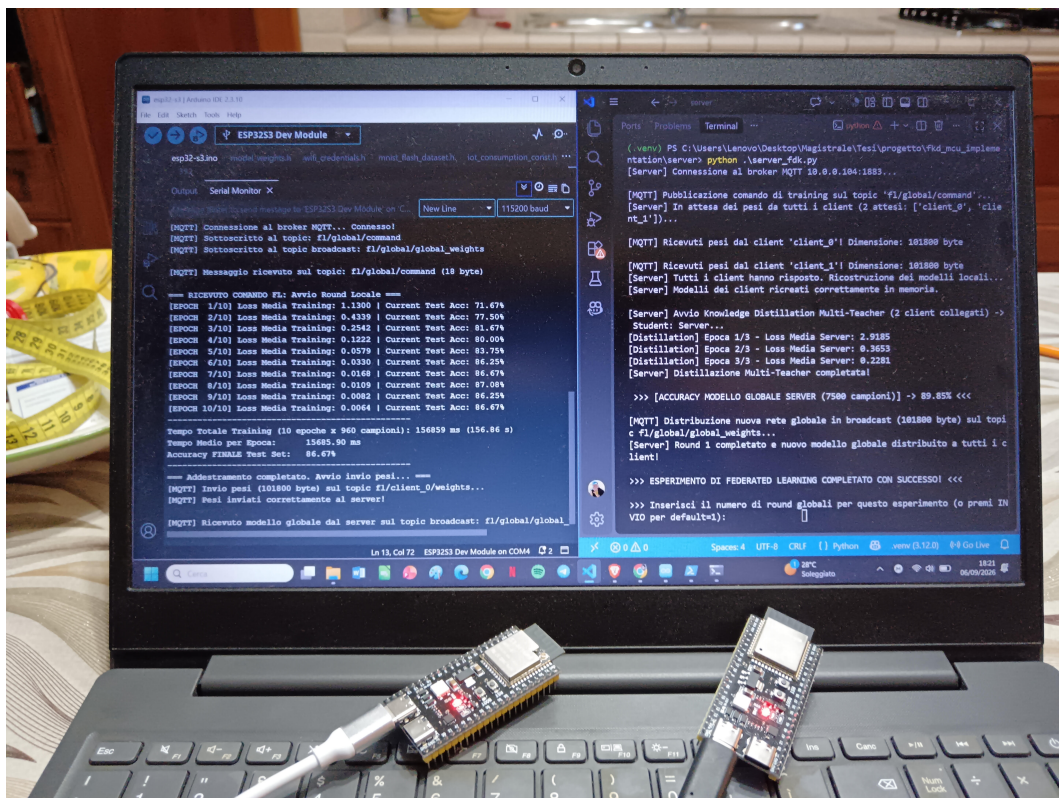


Figure VII.1: Physical implementation of FKD

VIII Conclusions

This thesis addressed the critical operational challenges of deploying Federated Learning (FL) across resource-constrained Internet of Things (IoT) and Edge AI ecosystems. The physical limitations of commodity microcontrollers (MCUs), governed by constrained volatile memory (RAM), limited compute budgets (FLOPs), and strict wireless communication bottlenecks, render standard, full-precision federated pipelines unsustainable for mass edge deployment.

To systematically investigate and quantify the multi-dimensional trade-offs between predictive accuracy, model footprint, and edge resource consumption, this work introduced *FedComp*: a modular, reproducible, and extensible empirical benchmarking framework built on top of the *Flower* ecosystem [13]. *FedComp* provides a standardized testbed that integrates and isolates four representative paradigms: a full-precision **Baseline FL**, **Quantization-Aware Training (QAT)**, **Federated Knowledge Distillation (FKD)**, and dynamic structured pruning via **Ordered Dropout (OD)**. Crucially, the framework embeds an analytical, literature-grounded profiling engine directly into the client runtime loop, enabling the granular tracking of communication bandwidth (B_{MB}), local inference FLOPs, and estimated energy expenditure (E_{total}) alongside traditional predictive metrics across both IID and severe non-IID data regimes ($\alpha = 0.1$, native FEMNIST).

The empirical investigation across MNIST, Fashion-MNIST, and FEMNIST under both IID and severe Non-IID regimes provided actionable quantitative insights into distributed model optimization:

- **Quantization-Aware Training (QAT)**: demonstrated the most effective communication trade-off, reducing network transmission overhead by 75% (1-byte INT8 versus 4-byte FP32) while matching full-precision baseline accuracy on standard vision tasks. However, the analysis also exposed its numerical fragility: under micro-batch regimes ($B = 1$) with large output spaces (FEMNIST 62 classes), standard QAT suffered

from gradient stagnation, requiring long training horizons to establish stable quantization scales;

- **Federated Knowledge Distillation (FKD):** emerged as the most robust architecture under extreme statistical heterogeneity. By decoupling local client training from server-side aggregation through an unlabelled proxy dataset, FKD prevented client drift and provided strong multi-class convergence with a lightweight backbone (`TinyNetIoT`), despite a minor top-1 accuracy penalty (1–7%) due to halved channel capacities;
- **Ordered Dropout (OD):** proved the feasibility of elastic width sub-sampling, dynamically instantiating nested sub-models. While delivering notable savings in local FLOPs and computational energy, evaluating multi-tier sub-networks aggregate-wide introduced an accuracy gap on complex multi-class manifolds due to restricted representational capacity in narrow slices.

The primary contribution of this thesis lies in bridging the gap between theoretical model compression and tangible edge resource profiling. By making *FedComp* open-source and ensuring determinism across partitioning regimes, this work provides the research community with actionable guidelines for architecting resource-aware federated systems. The findings demonstrate that no single compression technique dominates across all dimensions; rather, **QAT** represents the optimal solution when uplink bandwidth is the primary bottleneck, whereas **FKD** provides the superior trade-off when statistical heterogeneity and on-device memory limits dominate.

The modular nature of the *FedComp* architecture opens several research directions for future exploration:

1. **Hybrid Compression Pipelines:** investigating the synergistic combination of complementary techniques, such as applying Quantization-Aware Training directly onto the dynamic sub-networks generated by Ordered Dropout, or distilling quantized teacher representations;
2. **Asymmetric and Dynamic Client Allocation:** extending the static identifier-based profiling ($p \in \{0.25, 0.5, 0.75, 1.0\}$) into an adaptive, runtime scheduling mechanism where clients select compression rates dynamically based on instantaneous battery levels, thermal states, or wireless signal strength;

3. **Broader Data Modalities and Topologies:** expanding the benchmarking suite beyond 2D computer vision to include 1D sensor time-series data (e.g., Human Activity Recognition, predictive maintenance) and decentralized Peer-to-Peer (P2P) federated topologies.

Bibliography

- [1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In Aarti Singh and Jerry Zhu, editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282. PMLR, 20–22 Apr 2017. 1, 3, 10, 11
- [2] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D’Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badi Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaid Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrede Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Mariana Raykova, Hang Qi, Daniel Ramage, Ramesh Raskar, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. Advances and open problems in federated learning, 2021. 1, 12
- [3] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 1, 7, 47
- [4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>. 1, 7, 8, 9

- [5] Lei Deng, Guoqi Li, Song Han, Luping Shi, and Yuan Xie. Model compression and hardware acceleration for neural networks: A comprehensive survey. *Proceedings of the IEEE*, 108(4):485–532, 2020. 1, 20, 21, 23, 25
- [6] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper, 2018. 2, 3, 17, 23
- [7] Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. In D. Touretzky, editor, *Advances in Neural Information Processing Systems*, volume 2. Morgan-Kaufmann, 1989. 2, 16
- [8] Samuel Horvath, Stefanos Laskaridis, Mario Almeida, Ilias Leontiadis, Stylianos I. Venieris, and Nicholas D. Lane. Fjord: Fair and accurate federated learning under heterogeneous targets with ordered dropout, 2022. 2, 3, 17, 20, 23, 24, 25, 27
- [9] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015. 2, 3, 15, 20
- [10] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013. 2, 3, 18, 66
- [11] Darryl D. Lin, Sachin S. Talathi, and V. Sreekanth Annapureddy. Fixed point quantization of deep convolutional networks, 2016. 2, 66
- [12] Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets, 2019. 2, 66
- [13] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. Flower: A friendly federated learning research framework, 2022. 2, 26, 46, 90
- [14] Tao Lin, Lingjing Kong, Sebastian U. Stich, and Martin Jaggi. Ensemble distillation for robust model fusion in federated learning, 2021. 3, 21
- [15] Mark Horowitz. 1.1 computing’s energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14, 2014. 3, 31, 50

- [16] Buchanagandi E. Nyamajeje and Lawrence Kerefu. A secure and edge-aware bandwidth optimization model for cost-effective offloading in mobile cloud computing. *Concurrent Engineering: Research and Applications*, 2025. 3, 31
- [17] Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, 2, 2010. 3, 45
- [18] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *CoRR*, abs/1708.07747, 2017. 3, 46
- [19] Sebastian Caldas, Peter Wu, Tian Li, Jakub Konečný, H. Brendan McMahan, Virginia Smith, and Ameet Talwalkar. LEAF: A benchmark for federated settings. *CoRR*, abs/1812.01097, 2018. 3, 4, 46
- [20] Enmao Diao, Jie Ding, and Vahid Tarokh. Heterofl: Computation and communication efficient federated learning for heterogeneous clients, 2021. 20, 21, 23, 24
- [21] Ying Zhang, Tao Xiang, Timothy M. Hospedales, and Huchuan Lu. Deep mutual learning, 2017. 20
- [22] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding, 2016. 23
- [23] Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. Pruning convolutional neural networks for resource efficient inference, 2017. 24, 25
- [24] Muneer Bani Yassein, Mohammed Q. Shatnawi, Shadi Aljwarneh, and Razan Al-Hatmi. Internet of things: Survey and open issues of mqtt protocol. In *2017 International Conference on Engineering & MIS (ICEMIS)*, pages 1–6, 2017. 75

Ringraziamenti

Ed eccomi qua, alla fine del percorso... devo dire che me la sono goduta. Non ho mai sofferto troppo l'ambiente universitario, anzi, mi ha dato tanto come ad esempio nuove amicizie ed esperienze e forse mi ha fatto capire cosa voglio fare nella mia vita professionale. Per questo ci tengo a ringraziare delle persone che sono state importanti in questo percorso.

Il primo ringraziamento va al prof. Ficco e al dott. Guerriero. Lavorare a questo progetto è stata un'esperienza che mi ha insegnato tanto. Ricorderò con nostalgia gli incontri nel LabIoT dove io ed Antonio battibeccavamo simpaticamente disegnando sulla lavagna modelli (disegnati male) e le varie problematiche del progetto. Non ho mai vinto una discussione tecnica con te, ma devo dire che sono state davvero stimolanti e per questo ti ringrazio. Grazie anche al prof. Ficco, è stato un'insegnante molto bravo, ma la cosa che ho apprezzato di più è stata la sua simpatia e capacità di mettere a suo agio chiunque si trovasse con lui.

Mamma e papà, anche a voi devo dire grazie. Siete stati sempre presenti nel momento del bisogno, specie in quest'ultimo periodo che non è stato semplicissimo. Ho sempre creduto che tenere certe cose per me fosse la cosa migliore, però parlare con voi, per quanto a volte possiamo avere visioni diverse, è la cosa giusta da fare, perchè è quello che una famiglia deve fare. Mamma tu mi hai insegnato ad avere dei sogni e a gettare il cuore oltre l'ostacolo quando necessario. Papà, da te ho imparato che anche se le cose non vanno sempre come vorremmo, c'è sempre un'altra strada, una nuova possibilità, basta volerlo. Vi voglio bene.

Gianmaria, oltre che a mio fratello sei il mio migliore amico. Continueremo ad essere diversi ma anche simili su certe cose e devo dire che il modo con cui affronti le cose che accadono è la cosa che ammiro di più. Adesso per te si sta avvicinando il momento di prendere delle decisioni per il tuo futuro, ma sono sicuro che saprai cavartela e che ti realizzerai; d'altronde sei mio fratello perciò qualcosa di buono ce l'hai muahahahahaha (*risata malefica*).

Ai miei nonni Antonio e Antonietta. Sono stato così fortunato da avere due coppie di genitori. Nonno, avrei voluto più di ogni altra cosa che tu fossi qui oggi, saresti stato fiero di me, ma di certo non ho mai avuto bisogno di una laurea per essere un tuo motivo di orgoglio. Cose serie a parte speriamo che questo sia l'anno buono per far tornare lo scudetto a Torino, "addò adda sta". Nonna, sei una colonna portante nella vita di molti di noi, non ci hai mai fatto mancare niente e la tua forza combattiva è una fonte di ispirazione. Detto questo spero di mangiare le tue "cutulette" per ancora tanto tempo.

Zio Gigi e zia Pia, anche per voi non posso che spendere che parole di ringraziamento. Siete stati sempre di supporto e pronti a sollevarmi il morale quando serviva, ma anche critici con la giusta dose (zia penso che adesso mi vesto meglio eh). E' bello far parte di una famiglia unita e per distacco siete i miei zii preferiti.

Dalia e Fabio, voi siete dei fratelli più che cugini. Siamo cresciuti sotto lo stesso tetto e non potrei desiderare dei cugini migliori di voi. Negli ultimi anni ho seguito con grande passione le vostre partite e questo ha riacceso in me la voglia di mettermi in gioco e tornare a fare sport sia per staccare la spina dallo studio o dal lavoro che per far parte di un gruppo. Anche per voi non ho dubbi, siete il duo più chill che conosca e saprete realizzarvi nella vita. Restate comunque dei piedimontesi fake.

Grazie anche alla mia bis-nonna Maria, sei un esempio di longevità e di bontà. Per favore però, quando fai da mangiare, metti meno aglio.

Un grazie va anche agli altri miei nonni Vincenzo e Pina. Anche per te nonno, avrei preferito fossi qui ma va bene così... Mi hai trasmesso un inguaribile ottimismo (si so che non sembra ma lo sono) e te ne sarò sempre grato. Nonna anche se non riusciamo a vederci spesso per la distanza, sappi che ti penso spesso e ti voglio bene.

Grazie a tutti gli altri componenti della mia famiglia, non vi cito perchè sennò non ne usciamo più, ma se state sentendo o leggendo queste parole allora siete in mia compagnia e questo è più che abbastanza perchè vi voglia bene.

Bene ora passiamo agli amici e credo ci vorrà un pò, dopotutto sono uno popolare io...

Ai miei coinquilini Ernesto e Pierluigi, nonchè amici, nonchè compagni di liceo, nonchè compagni di merenda ah no quello era il Pacciani. Ernesto sei il mio bro, ci conosciamo da qualcosa come 20-21 anni, con te è sempre stato semplice parlare di tutto e sarai sempre il tipo più chill che conosca.

Certo sei anche un pò mongoloide perchè veramente non so come si possa preferire guardare la Kings League anzichè giocare a bowling. Pier sei un compagno di stanza fantastico e non vedo l'ora di continuare le nostre lunuunge pause caffè condite da discorsi che svariavano dalla politica ad aneddoti, dall'attualità alle relazioni, da One Piece a... One Piece. Daje che per quando facciamo 30 anni forse finisce.

A Raffaele e Gaetano, che dire... Quando non parlate dell'Inter siete pure simpatici. Scherzi a parte siete sempre i primi ad interessarvi alle mie novità e a supportarmi quando le cose non vanno al meglio. Sono sicuro che anche tra tanti anni le cose tra noi resteranno così come sono perchè il tempo o gli impegni non potranno indebolire questo legame.

A Giovanni O., un mix di entusiasmo, fedeltà, sicurezza e forse un pò di sana follia. Anche con te ci conosciamo da 20 anni e sei una persona in grado di trasmettere una gioia unica. Passare il tempo con te è davvero un'esperienza, che va dal passare un tranquillo pomeriggio a casa, al chiedere indicazioni alle due di notte ad una "signorina". No sul serio cercavamo di raggiungere l'ospedale di Benevento non fate pensieri strani.

Ai due Lorenzo, praticamente due persone diametralmente opposte. Ci siamo conosciuti durante il terzo anno e siete davvero i migliori compagni universitari che potessi trovare. Lorenzo Sc., sei davvero un'ispirazione, insomma sei quasi sposato, posto fisso, re indiscusso del relax però sotto sotto sai che sei più scarso di me a calcio. Lorenzo So., potrei dire un sacco di cose brutte su di te... però me le tengo per qualche ricatto. Siamo molto diversi e credo che sia proprio questo che ha permesso di avere un rapporto così bello con te. Anche tu sei una specie di modello per me, sia dal punto di vista professionale che per cose più serie. Conoscendoti posso dire di essere maturato ancora di più e per questo ti ringrazio. Devo dire che sotto sotto è bello avere uno che ti scrocca la casa.

Ad Alessandro, Giovanni D.R., Marcellino e Simone, siete davvero degli ottimi amici.

A Carlo, siamo passati dal tirati il registro di classe in testa al passare ore al Moonshine per la classica birretta. Con te i discorsi non sono mai banali e sono davvero contento di essere tuo amico.

A tutti i ragazzi del liceo, oltre ai già menzionati Ernesto, Pierluigi e Carlo ci sono Luigi, Gianluigi e i due Angelo. Ogni volta che ci si vede è come se non fosse passato neanche un giorno, siete davvero speciali. Si lo so devo organizzare la classica e sofferta pizzata.

A Giacomo, Amedeo e Rocco, amici e compagni di questa magistrale. Ricorderò con nostalgia le lezioni e le chiacchierate tra i banchi.

A Davide, Vincenzo, Simone ed Aurelio. L'ultimo semestre è stato particolarmente bello anche grazie alle ore passate in palestra a giocare a pallavolo e proprio in quei momenti ho trovato dei nuovi amici, vi sono grato.

Un grazie va alla musica di Mark Knopfler, i Dire Straits, De Andrè, Eric Clapton e tutti gli altri artisti che hanno reso gli interminabili viaggi Alife-Fisciano meno noiosi del solito.

L'ultimo ringraziamento va a te Chiara. Chi l'avrebbe mai detto che ci saremmo conosciuti alla fine dei corsi, praticamente quando tutti vanno via. Quella brava in matematica sei tu quindi lascio a te il calcolo delle probabilità. Quello che volevo dirti te l'ho già detto e sono contento di averlo fatto. Ho imparato che la vita è imprevedibile e indipendentemente da come è andata, sono stato fortunato ad averti nella mia vita. *Ci ho guadagnato il colore del grano.*